

Lab Manual

Contents

- Welcome to BIPN 145
- Modeling Neural Membranes
- String Nervous Systems
- Earthworm Electrophysiology
- Leech Electrophysiology
- *Drosophila* Behavior & Optogenetics
- Measuring Alpha Wave Activity Using EEG

Welcome to BIPN 145

This is a course about how we study the nervous system. You'll be asked to think, plan, and write like a neuroscientist.

This lab manual describes the set of experiments you'll be conducting over the course of the quarter.

This course is for everyone.

Neuroscience is a diverse discipline, with folks from many different academic and personal backgrounds. We'll be working together to create an equitable and inclusive environment of mutual respect, in which we all feel comfortable to share our moments of confusion, ask questions, and challenge our understanding.

Everyone should be able to succeed in this course. If you do not feel that is the case, please let us know.

Modeling Neural Membranes

The brain as a circuit board

We often describe the brain in terms of circuits, which is a useful and pretty accurate metaphor. In our Modeling Neural Circuits lab, you'll encounter two different ways to model the brain: using an online simulator and with a physical simulation on a breadboard. Both use the fundamental concepts of how electricity flows through substances to generate a working model of the biophysical properties of a neuron.

This introductory page will provide some background information to help you navigate the concepts of this lab.

Modeling resistance

A resistor converts electrical energy into heat when current flows through it. A current flowing through a resistance creates a voltage drop across that resistor, described by Ohm's law:

Ohm's Law

$$V = IR$$

Symbol	Meaning
V	voltage (Volts)
I	current (Amps)
R	resistance (Ohms, Ω)

In a neuron, the resistance is determined by the opening and closing of ion channels, which allow current to flow in and out of the cell. When more ion channels are open, more ions are able to flow. Ion channels can be represented with resistors in an RC circuit, since they resist the flow of electricity.

It's important to note the units in the equation above: 1 Amp flowing through a 1 Ohm resistor produces 1 Volt across the resistor. In electrophysiology, we typically work with different scales of numbers:

Unit	Equivalent
millivolts (mV)	10^{-3} V
nanoamps (nA)	10^{-9} A
megaohms (M Ω)	10^6 Ω

Sometimes you'll see Ohm's Law with conductance instead of resistance. Conductance (measured in siemens and often denoted by g) is the inverse of resistance. In other words, it's equal to $1/R$.

Kirchhoff's circuit laws

Kirchhoff's Laws

Kirchhoff's laws state two relatively intuitive facts:

1. **The voltage at a given point in a circuit is uniquely defined.** Thus, if there are two (or more) paths to go from point A in a circuit to point B, the voltage drop along each of those paths is the same.
2. **The voltage drops around a closed loop must sum to zero.** The sum of all currents going into a point equals the sum of all currents going out of it.

Modeling capacitance

Capacitors are defined by two conducting substances separated by a non-conducting substance in the middle. Capacitors accumulate electric charge on the insulator. The accumulated charge creates an electric field between the conductors and stores electrical energy. Salty fluids that make up the intra- and extracellular solutions of the cell are conductive, the lipid bilayer is not. Therefore, the neural membrane acts as a capacitor.

To summarize and add a couple useful symbols:

Circuit component	Symbol	A current step (I) like this...	... produces this voltage (V) response.	Equation
Resistor				$V = I R$ R is measured in ohms (Ω)
Capacitor				$I = C \, dV/dt$ C is measured in farads (F)

Modeling concentration gradients

In a neuron, there's also a steep concentration gradient due to the different concentrations of ion on either side of its membrane. This is the basis of the neuron's resting potential. The inside of the cell is more negative than the extracellular cell space – typically by about -60 mV, depending on the cell type. We'll use batteries to model this offset of voltage.

Putting it all together

With these components – resistors, capacitors, and a battery – current flow in a neuron can be modeled. In reality, there are many different ion channels, and multiple different ion conductances. We'll simplify each of these into one membrane resistance, and we'll model the membrane of the cell with one capacitor.

When we send current into a circuit with a capacitor, the capacitor will charge. In both a neuron and an electrical circuit, voltage rises to a steady state level asymptotically in the response to an external current (e.g., from an action potential or passively passing charge). Once the external current is shut off, the voltage drops in a similar asymptotic way. As you'll discover today, this charging time course depends on a few different factors.

These characteristics of the membrane voltage can be measured with a simple voltmeter in the RC circuit. Opening more ion channels (or resistors, as in this RC circuit), which decreases resistance and increases conductance, will express a more rapid increase in voltage on the voltmeter, visually demonstrating the properties of resistance in a neuron. We can calculate a time constant, which represents the time it takes to reach a steady state after a change in voltage across the membrane:

i Time Constant

$$\tau = RC$$

Symbol	Meaning
τ	time constant (seconds)
R	resistance (ohms)
C	capacitance (farads)

Breadboards & electronics are useful tools for neuroscience research

Lastly, these types of circuits are really useful for designing neuroscience experiments. Many labs rely on breadboards and microcontrollers (e.g. Arduino) to create custom setups. Once you know how to wire up a circuit and program an Arduino, you can do pretty much anything.

[This video](#) will explain how to complete the activities in our lab on a breadboard.

Neuromembrane

About Neuromembrane

Neuromembrane was made by a team of University of Alberta researchers and a company known as Atmist. Many thanks to Christelle Sabatier, Michael Wright, and Declan Ali for sharing similar lesson plans. Their ideas are integrated here.

Part I: Resting membrane potential

First, we'll explore the properties of the membrane that lead to a resting membrane potential. By the end of this part, you will be able to:

- Explain how a negative resting membrane potential is maintained
- Identify the factors that determine and maintain resting membrane potential
- Use the Goldman-Hodgkin-Katz to calculate the membrane potential given the concentrations and permeabilities of K^+ and Na^+
- Generate simulation data to illustrate the dependence of the resting membrane potential on K^+

Protocol

1. Go to <https://neuromembrane.biology.ualberta.ca/> and click "Continue to App."
2. To open the resting membrane potential simulation, click on the top left to open a menu of stimulations. Choose "Resting Potential Simulation."

On the right hand side, you'll see a membrane. The area below the membrane is the inside of the cell. The area above the membrane is the outside of the cell.

3. This simulation will open by default without any leak channels or pumps. Let's first add a couple of leak channels to the membrane and observe what happens. Click the plus sign next to "LEAK CHANNELS" to add a Na^+ and K^+ channel to the membrane.

Reflection

These single channels represent many, many channels in the membrane that are allowing either Na^+ or K^+ to cross. By default, these will open with a relative permeability of 0.05:1 ($Na:K$). Note that at these default settings, there is also equal $[Na^+]$ and $[K^+]$ inside and outside of the membrane.

4. First, we'll observe what happens over time when we have more K^+ permeability, but equal ion concentrations. **Before running the simulation, write a 1-2 sentence prediction for what will happen in Table 1.**
5. Once you have your prediction, click on the play button to run the simulation. You can also change the speed so that it will run faster. **What happens to the voltage over time? Why is the voltage across the membrane this value? Write your observations in Table 1.**
6. In order to modify the concentration across the membrane, we'll need a Na^+/K^+ pump. Click the plus button next to Na^+/K^+ pump to add one to the membrane.

7. Notice that the CONCENTRATION SETTINGS have now changed as well. Change these concentrations back to the previous settings — where all of the concentrations are 50 mM. **Before running Simulation #2, write your 1-2 sentence prediction in Table 1.**
8. Run Simulation #2 and observe what happens to the membrane potential. **Has adding a Na⁺/K⁺ pump changed anything about the resting membrane potential? Why or why not? Add your observations to the table.** For this simulation, note that we're just using the Na/K pump to be able to change the concentration across the membrane. The Na/K is not operational (yet).
9. Go back to the settings page. In real neurons, ion concentrations are not equivalent. Change the concentrations of K⁺ and Na⁺ back to their default values (Na: 10 mM inside, 140 mM outside; K: 140 mM inside, 4 mM outside) to reflect more biological values across the membrane. **Write your prediction for Simulation #3 in the table.**
10. Run the simulation again and observe what happens to the membrane potential.



Log your predictions

What happens to the voltage over time? Why is the voltage across the membrane this value? Write your observations in a table formatted to one like Table 1, below. Here is a [Google Spreadsheet Template](#).

Table 1. Simulation predictions & observations

Sim #	Types of Channels	[Ion]	PNa/K	Prediction	Observation
1	Leak channels only	All 50 mM	5 Na ⁺ ; 100 K ⁺		
2	Leak channels & Na ⁺ /K ⁺ pump	All 50 mM	5 Na ⁺ ; 100 K ⁺		
3	Leak channels & Na ⁺ /K ⁺ pump	Na ⁺ high outside; K ⁺ high inside	5 Na ⁺ ; 100 K ⁺		
4	Leak channels & Na ⁺ /K ⁺ pump	Na ⁺ high outside; K ⁺ high inside	100 Na ⁺ ; 5 K ⁺		
5	Leak channels & Na ⁺ /K ⁺ pump	Na ⁺ high outside; K ⁺ high inside	100 Na ⁺ ; 100 K ⁺		

11. Using the Goldman-Hodgkin-Katz (GHK) equation for Na⁺ and K⁺ only, calculate what the voltage across the membrane should be for simulation #3 and show your work. Does this match the voltage in the simulation? **Answer Q1 on the quiz. Report your answer in mV.**

Here are a few hints to help you:

- P is relative permeability; other constants can be found on the lecture slides.
- To simplify, you may determine a constant for the first portion of the equation (RT/F) and use that for future calculations.

- You can use “Insert > Equation” in Google Docs or Microsoft Word to show your work, or simply jot it on paper, take a picture, and paste it here.
- Google search has a built-in scientific calculator! You can bring it up by typing any formula into the search bar. For example, type `=5*150` into Google Search.

Q1. Calculation of GHK

Using the GHK equation for Na^+ and K^+ only, calculate the voltage across the membrane for simulation #3 and show your work. Does this match the voltage in the simulation? Report your answer in mV.

12. Finally, let's check to see how much ion permeability matters. Go back to the settings page and reverse the Na^+ and K^+ permeabilities.
13. Write your prediction for Simulation #4 in Table 1.
14. Run Simulation #4 and observe what happens. Record your observations in Table 1 and respond to the question below. **Answer Q2 on the quiz. Report your answer in mV.**

Q2.

With the high permeability to Na^+ and low permeability to K^+ , what is the resting membrane potential? Report your value in mV out to one decimal point.

15. On the settings page, set both the Na^+ and K^+ permeabilities to 100. Write your prediction in Table 1.
16. Run the simulation to see what happens, record your observations, and respond to the question below. **Answer Q3 on the quiz.**

Q3.

Which of these simulations best models a typical neural membrane?

17. Next, we'll check to see whether or not the membrane potential depends on $[\text{K}^+]_{\text{out}}$. Reset the permeabilities and ion concentrations to the default values.
18. Systematically change $[\text{K}^+]_{\text{out}}$ to fill in the table below. Hint: if you hover over the chart on the left, you'll be able to see the precise voltage.

Q4

Complete Table 2 and then answer Q4 on the quiz. [You can use the Table 2 tab on the Google Spreadsheet template.](#)

Table 2. Dependence of V_{rest} on $[\text{K}^+]_{\text{out}}$

[K ⁺]out (mM)	Vrest (mV)
4	
10	
20	
50	
200	

19. Click the "Toggle Circuit Diagram" button on the top right hand corner to overlay circuit components.

Q5.

In a few words, describe what each of the following are modeling in the neuron. The first one has been done for you. Once you're done, answer Q5 on the quiz.

Component	What it models
I_Na	sodium current across the membrane
g_Na	
E_Na	
I_K	
g_K	
E_K	

Part II: Passive membrane simulation

In this part, you'll set up a simulation that models one portion of an axon or dendrite. By the end of this part, you will be able to:

- Describe how voltage passively spreads through an axon or dendrite
- Understand length and time constants
- Determine how the passive spread of voltage is affected by diameter and membrane capacitance

Protocol

1. Go to <https://neuromembrane.biology.ualberta.ca/> (if you're not already there) and open up the "Passive Membrane Simulation" (or Cable Theory Simulation) in the menu in the top left corner.
2. Add an external recording electrode that is two length constants (λ) away from your stimulating electrode by clicking the **+** button and changing the value after the second recording site. **Answer Q6 on the quiz.**
3. Inspect the parameters on the left hand side of the screen. To see how these electrical components are being modeled, click on **3D** on the bottom to toggle to a **2D** diagram, and then choose "Show Circuit Diagram." **Answer Q7 on the quiz.**
4. Re-set the cable settings and click on "CREATE SIMULATION" to run our first iteration of this passive current injection. Observe the top plot to fill out row one in Table 3. For your "observations," take a close look at the Voltage vs. Time plot.

Reflection

Why is our current injection this shape when recorded across the membrane?

Table 3. Current injection results

Sim #	Diameter	Cm	Peak voltage at 2λ	Distance to 2λ electrode	Voltage at 0λ electrode @ 28 ms	Voltage at 2λ electrode @ 28 ms	Observations
1	5 μm	1					
2	2 μm	1					
3	2 μm	2					

5. The dorsal root ganglion axon of a giant blue whale has a diameter of $\sim 2 \mu\text{m}$. In the CABLE SETTINGS on the left, change the diameter of the simulated axon to model this axon.

Reflection

With this smaller diameter of $2 \mu\text{m}$, do other parameters of the membrane (bottom of the CABLE SETTINGS window) change? If so, why?

6. Run the simulation with the smaller diameter and observe the Voltage vs Time plot again. Hint: you can use the Autoscale button () if the trace goes off the plot. Fill out Table 3 for this $2 \mu\text{m}$ cable simulation.
7. Double the specific capacitance of the membrane, from 1 to $2 \mu\text{F}/\text{cm}^2$. Describe what happens to the time constant and the resulting voltage over time plot with more capacitance in Table 3.

Optional: Later in the course, you may want to use the Neuromembrane simulator to model the action potential. There is a [protocol here](#).

RC Circuits

Let's build a circuit that models the passive membrane simulations you performed in the last part of our **Modeling Neural Membranes** lab.

Parts	Cables & Adapters
PowerLab & Computer	DIN8 to BNC adapter (1)
1 μF capacitor	BNC to double banana adapter (1)
Resistors: 3 k Ω , 15 k Ω , 33 k Ω , 100 k Ω , 220 k Ω , 330 k Ω	Banana plug cable (4)
	Alligator clips (4)

Getting familiar with your breadboard

Breadboards are really useful ways to prototype and model electrical circuits (and in our case, neural circuits). Some holes on the breadboard are connected together, so putting wires into connected holes provides an electrical contact between them. These connections are organized with two main rails (rows) on the sides, typically used for power, and many columns. Knowing how these components are connected is crucial to understanding [how to use a breadboard](#).

Building an RC Circuit

Build the circuit below. We'll use the OUTPUT of your Powerlab to send a pulse of current through the circuit.

1. It helps to start with the output ends in the power rails and work your way around. The **red** wire from the PowerLab is the positive terminal.

CAUTION: Do not connect the + and - terminals. This will short the circuit, heat, and melt the plastic. Check that your circuit is set up properly before turning on the pulse from your computer.

2. Attach single BNC adapters to the + and - outputs of the PowerLab.
3. Attach a red banana cable to the + terminal and a black banana cable to the - terminal of your PowerLab.
4. Attach the banana cables to red and black jumper wires using alligator clips, and insert the black and red jumper wires to the power rails, as shown in the circuit.

NOTE: Ensure that all circuit components are properly inserted into the breadboard. The wires should make contact with the metal strips inside the breadboard.

5. Add additional jumper wires, a resistor, and a capacitor, as shown in the circuit.
6. We'll use the PowerLab as our voltmeter, to record voltage in our circuit. Plug a **BNC to DIN8 adapter** and **double banana adapter** into Input 1 of the PowerLab.
7. Connect the ground/reference (**black**, -) side of the double banana adapter to the circuit ground with alligator clips.

NOTE: The "ground" in this circuit is really going to be our reference electrode. It should be connected to the black side of Input 1.

8. Connect the "hot" pin (**red, +**) of the double banana adaptor to the "record here" point on the circuit diagram.
9. Open LabChart with the **Circuit Lab** settings (<http://bit.ly/labchart>)
10. Set up the Stimulator to stimulate with 1 V for **1 second**. Make sure the range of your recording channel is 1 V. Switch the stimulator to "on".
11. Open Scope View and press **>Start**. You should see a waveform on your screen.

Autoscale!

You may need to Autoscale the channel (right click) to be able to see the entire trace.

Q8.

The time constant should equal $\tau = RC$. What *should* the time constant be (in seconds), given the 100 kOhm resistor and 1 uF capacitor in this circuit? Mind your units. **Answer Q8 on the quiz.**

Measure your time constant

We can also measure the time constant by measuring how long it takes the curve to rise or decay. One time constant (τ) is defined by when the circuit rises to **63.2%** of its total charge (alternately, we could see how long it takes to decay $1/e$, about 37%).

To empirically measure your time constant, follow these steps.

1. Make sure your voltage is very close to zero at baseline.
2. Find the maximum voltage (V_{max}) that your circuit reaches.
3. Calculate $0.632 * V_{max}$
4. Using the marker tool (right click anywhere in the window and choose "Clamp to trace") find the point in your trace that is equal to the value calculated in Step 3.
5. Move your cursor to the start of the stimulus. In the upper right corner of the window, you'll see something that says Δt . This is the change in time since the mark on the curve and your time constant.

From theory to observation

What is your recorded time constant (τ) with 1 V stimulation, a 100 k Ω resistor, and a 1 uF capacitor? How close is the recorded time constant to the value you calculated with $\tau = RC$?

In engineering, 5τ is considered the time it takes for a capacitor to fully charge (technically, its at 99% of full charge). The time constant is symmetrical for charging as well as losing charge. **How long would it take this patch of membrane to return to baseline, then? What does that mean for *computation* happening in this membrane?**

Record and plot your time constant with different resistors

Next, we'll see how the time constant of the circuit changes in response to the resistance strength. Use the following resistors in your circuit, and use the steps above to determine the time constant for each.

****Table 4. Resistance vs. Time Constant**

Resistance	Time Constant
3 k Ω	
15 k Ω	
33 k Ω	
100 k Ω	
220 k Ω	
330 k Ω	

Plot your results

Plot the time constant against the resistance. Answer Q9a and Q9b on the quiz. Look for the corresponding tab on the [Modeling Neural Membranes Google Sheet template](#).

Plotting Tips

- **Google Sheets:** Use the template provided above and use **Insert > Chart > Scatter**.
 - Add a trendline under **Customize > Series > Trendline**.
 - Make sure that none of the markers for your points are cut off. If they are, change your axes so that they fit.
 - Enter your values into the table *without* units so that Sheets will recognize them as numerical. You can indicate the correct units in the table and axes labels.
- **Python:** Use the [Plot Scatter](#) notebook!

Check your time constant against the literature

Let's circle back to biology. Are the time constants you measured similar to what you'd find in a real neuron? Check this website to look at a collection of examples from the literature: https://neuroelectro.org/ephys_prop/index.html

Choose a paper from the example above and look for the reported time constant. Is the time constant in your circuit similar? If not, why might it be different?

Troubleshooting

Observation	Likely Issue	Possible Solution
There isn't any voltage in your circuit	The power terminals are not connected properly Your stimulator is not on	Make sure everything is connected correctly on your breadboard. Consult https://wiring.org.co/learning/tutorials/breadboard/index.html Check that the stimulator panel is switched on, and that you pressed Start
Your signal is really noisy	You're not grounded. Your pulse is not being properly sent.	Double check your circuit is connected correctly
You don't see a fin-shaped waveform	Your Scope/Chart View is not scaled properly. Your capacitor is not properly connected or broken.	Try autoscaling Check the capacitor placement or replace it.
You've got "rabbit ears" on your waveform	Your voltage is reversed somewhere	Check the power terminals are connected in the right direction

String Nervous Systems

Background & Goals

Why are we recording from a string?

The purpose of this lab is to get familiar with the recording equipment and software that we'll be using in this course.

First, we'll learn how you can configure LabChart 8 to acquire data and send out pulses of electricity. Then, you'll record the **stimulus artifact** (the passively conducted signal from the stimulus) from a saline-soaked piece of string. The string mimics some of the properties of the earthworm that you'll observe in a future experiment. This will give you the opportunity to learn how to use the stimulating and recording devices on the PowerLab, as well as reduce noise and stimulus artifacts from your recordings, before you actually have to do all of this in an experiment.

Asking questions during this lab is a great idea, and will help you succeed in future experiments.

Pre-Lab

Before the lab, watch this video and read the protocol below:

Today you will:

- Configure a recording experiment in LabChart 8.
- Determine how to record signals from a string.
- Identify sources of electrical noise in your rig.

This lab was adapted from Experiment #1 from the BIPN 105 & BIPN 145 Lab Manuals, as well as AD Instruments.

Protocol

Supplies	Cables & connectors	Solutions
String	BNC to single banana adapter (2)	Saline
Dissection tray	BNC to double banana adapter	
PowerLab 26T, connected to computer	BNC to DIN8 adapter	
BioAmp	Alligator clip adapters (2)	
Sewing pins (2)	Banana plug cords (2)	
Faraday Cage		
Large Petri Dish		
Eyedropper		

I. Setting up LabChart 8

LabChart 8 is the software we'll be using to record electrophysiology data. Before performing our experiment, we need to set up LabChart to **acquire** and **display** the data as we want.

1. Turn on the PowerLab 26T.
2. Open LabChart 8. You should see that the PowerLab is connected with a green check mark.
3. Open a new experiment by choosing the **New** button (bottom left-hand corner).
4. First, we need to decide how many channels to record. Go to **Setup > Channel Settings**.
 - At the bottom of the window, change the number of channels to **3**, and make sure that they are all on (box on the left is checked).
 - Rename the first channel "**Stimulus**" and the second channel "**Raw Recording**".
 - Set the range for Channel 1 (your "Stimulus" channel) to **5 V**.

Note: The range determines the range of values the PowerLab will record, and when using the BioAmp (later in this lab), it will actually change the amplification.

- Set the range for Channel 2 (your "Raw Recording" channel) to **100 mV**.
- Make the sampling rate for Channel 1 "**40k/s**" and ensure that "*Same sampling rate for all channels*" is checked.
- Make sure that the far-right column says "**No calculation**" for all three channels.

II. Finding sources of electrical noise

One of the biggest issues in electrophysiology is noise from overhead lights and other electronics in the room. It'll help to get an understanding of where noise in your rig is coming

from.

1. Connect a DIN8-BNC adapter and a BNC to double banana adapter to **Input 2** of the PowerLab. Plug in a red banana cable to the positive (red) side of the connector.
2. Hold the lead (the end of the banana cable) in your hand.
3. In the Channel Settings window, click on **Input Amplifier** for Channel 2. The Input Amplifier window will show the voltage that you're recording. This allows for precise setting of the input range for a recording channel and provides filtering options.
4. The signal at your rig may be variable. To adjust the sensitivity of the channel, choose an appropriate range setting from the **Range** drop-down list in the Input Amplifier dialog.

Note: The number displayed in the range menu indicates the maximum input voltage currently selected. Notice how as you decrease the range the vertical scale changes and the small rhythmic deflections that appear on the signal trace increase in amplitude.

5. Choose a range such that your signal fills about $\frac{2}{3}$ of the window.
6. Observe the change in the recorded voltage with the electrode lead held overhead (near the lights) and then near the ground (away from the lights).
7. Try the lead inside of your Faraday cage. Is the signal more or less noisy?

Tip: Is your Faraday cage actually grounded? How could you ground it?



Questions for reflection

- How does the signal change as you move the electrode lead around?
- What do these sources of noise mean for our recording configuration, and our attempts to record electrical activity?

III. Recording stimulator outputs

We'll use our PowerLab as a stimulator that can send voltage pulses into our specimen. We can change the timing, shape, amplitude, and repetition of these stimuli via the Stimulator window in LabChart. We'll directly record these stimuli by sending the output of the PowerLab into an input.

1. Change your view to **Scope view**, which looks like the sweeps of an oscilloscope. The advantage of Scope View is that it allows the overlay and averaging of your data.
2. In Scope view, set the **Duration** to **20 ms**. This changes how long each sweep of data is.
3. Remove the DIN8-BNC adapter & cable from Input 2.
4. On the PowerLab, connect the **(+) Output** to **Input 1** using the BNC cable and DIN8-BNC adapter.
5. Go to **Setup > Stimulator** to open the Stimulator window. This is the window where you can modify the shape, timing, and amplitude of the stimulus output.
6. Make sure that the **Waveform Name** is set to **Pulse**.
7. Change the range of the Pulse Height to **-5 to 5 V** by clicking the ruler button next to the Pulse Height option.

8. Configure the following by clicking and then editing the values in the window:

Parameter	Value
Start Delay	1 ms
Pulse Width	0.1 ms
Pulse Height	0.150 V

9. Close the Stimulator window.

10. Go to **Setup > Stimulator Panel** to open a smaller window where you can directly modify the stimulus. This window can stay open while you're recording.

11. Press **> Start** in the main LabChart GUI.

12. Increase the stimulator voltage (Pulse Height) and watch the level change in the window.

13. Try a negative stimulus voltage.

14. Open the Stimulator window again and vary the delay, duration, and waveform shape, and see what happens. If your recording is cut off, how can you fix it? (*Hint: see Visualization tips in Meet Your PowerLab.*)



Questions for reflection

- Why would you want to record the stimulus output?
- Why would you want to delay the stimulus output?
- Why would you want to send different shapes of waveforms?

IV. Recording the stimulus artifact

Lastly, we'll add an amplifier (the BioAmp) to our circuit, and use this to record from a string soaked in saline. The string doesn't have a nervous system, but we'll add some voltage with our stimulator and observe the change in voltage from another point on the string.

Set up your string stimulation experiment

Your setup for the string stimulation will look like this:

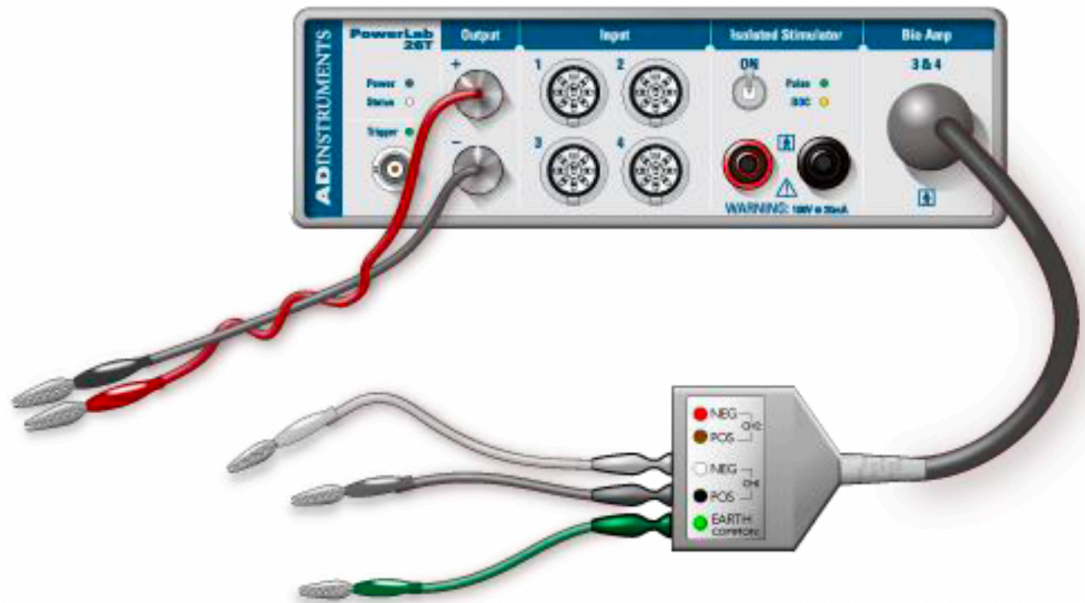


Image courtesy of ADInstruments ©

1. Remove all of the cords from the PowerLab.
2. Connect your two single BNC to banana cable adapters into the **(+)** and **(-)** Outputs of the PowerLab.

Note: The color of these does not matter, but it can help to be consistent — red for the anode (+ Output) and black for the cathode (- Output).

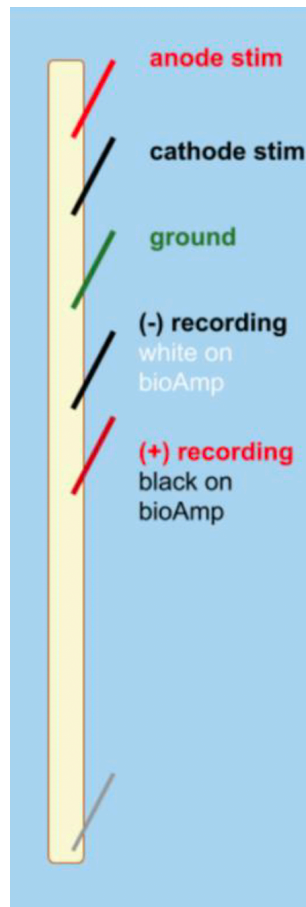
3. Using banana cables, connect the **(+) Output** to your **anode** stimulating electrode (the sewing pin) and the **(-) Output** to the **cathode** stimulating electrode.
4. Connect the BioAmp to the PowerLab.
5. Connect the (+) recording, (-) recording, and ground needle electrodes to your string and into the corresponding spots on the BioAmp.

Note: The colors on the BioAmp defy convention — consult the diagram above for clarity.

6. Change the configuration of your recording so that you are now using the BioAmp:
 - Go into **Setup > Channel Settings**.
 - Make sure that **BioAmp** is listed under the Input Amplifier column for Channel 3.
 - **Uncheck** Channel 2 — we don't need to record from it anymore.
 - Change the name of Channel 3 to **"BioAmp Recording."**
7. We've removed our ability to directly record the stimulus output, so we need another way to know when the stimulator sends outputs. Open the Stimulator Settings and set the **"Marker Channel"** to Ch1 (your "Stimulus" channel). With this setting, LabChart will put a small tick on Channel 1 to show you when the stimulus was sent.

Note: This marker tells you *when* the stimulus was sent, but does not indicate the height or length of the stimulus. This means you need to label your pages with your stimulation parameters.

Stimulate your string!



1. Place your string in saline in the small petri dish, and give it a few minutes to soak. If it's not completely soaked, you will not get a stimulus artifact.

Note: If your string dries out during the experiment, you can use an eyedropper to add saline to it.

2. Lay the string out on the dissection tray, with the stimulating and recording electrodes laid out as in the diagram above.
3. Set the stimulator voltage to **1 V**, and press **> Start**. You should see your stimulus recording on Channel 1, as you did before, but you should also now have a recording on Channel 3 (BioAmp Recording).
4. Change the scaling if you cannot see the entire deflection.
5. If you do not see anything, increase the voltage (no more than 5 V).

Note: This recorded deflection is called the **stimulus artifact**. This artifact can exceed the range of our amplifier. Recovery from the artifact should be rapid, however, so that the baseline is reached within one millisecond or so after the stimulus pulse.



Question for reflection

Is the amplitude of the stimulus artifact the same as the amplitude of the stimulus? Why or why not?

6. Increase the stimulator voltage, and stimulate again. What happens?

Note: Change the name of Scope pages, or make comments on the recording, to indicate what you've done during that run of the stimulus. You need to know the metadata for the experiment once you're ready to export.

7. Change the stimulus to a **negative voltage**, and stimulate again. Does the stimulus artifact change?

8. Choose one of your pages where you have a stimulus artifact with a clear peak. Using the **Marker tool**, measure the latency from the stimulus to the beginning of your artifact. How much of a delay is there? How much of a delay *should* there be? (How fast does an electrical current flow?)

9. Change the location of the stimulating electrodes. How does this change the shape of the artifact?

10. Remove the ground electrode. How does this change your recording?

Save your file & export your data

LabChart buffers files to disk in case of a power failure or computer crash, but it's wise to save work frequently. Saving files in LabChart is the same as saving any file on your personal computer.

Please backup your LabChart data files to the Google Drive you set up at the start of the quarter.

1. Save the **settings file** for your experiment by going to **File > Save As Settings File** in LabChart. Save it in a safe place (not locally on the computer). You will need this settings file for the earthworm experiment.
2. Save the **data file** for your experiment by going to **File > Save As** and save as a LabChart Data File (.adicht).
3. Go to the Scope page for **two of your favorite string stimulation experiments** that you can compare and contrast — for example, two different voltages, or with two different distances between the stimulator and recording electrodes.
4. Follow [these instructions for exporting your data](#).
5. Plot your recordings (separately or on the same axes — be sure to label two different recordings if overlaid). Make sure your axes are labeled with units. You can create these plots in [Python](#) or [Excel](#).
6. Copy your plots into a Word or Google document.
7. See your instructor's Canvas assignment for detailed directions about what else you need to submit.

Troubleshooting

Observation	Likely issue	Possible solution
You can see the stimulus marker on Channel 1, but no stimulus artifact on Channel 3	Your string may not be soaked in saline well enough	Add saline to the string
The recording trace is cut off	Your visualization needs to be re-scaled, or your range is too small	Right-click on channel and choose Auto Scale (see Visualization tips above); change the range on your recording channel
Your string recording trace has clear 60 Hz noise	You're not grounded, or there are ground loops	Make sure your Faraday cage and ground electrode are both grounded to the back of the PowerLab. If this doesn't help, consult an instructor.

Earthworm Electrophysiology

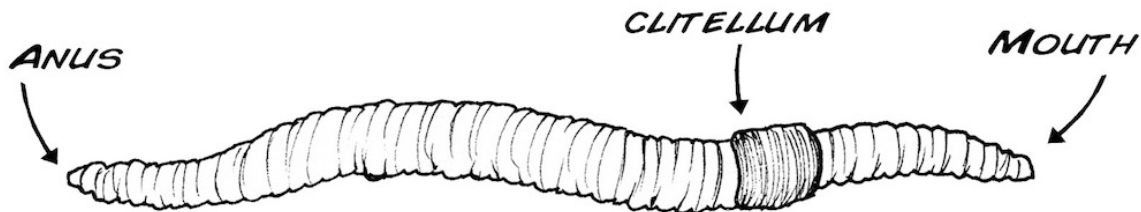
Background & Goals

Earthworm essentials

Earthworms are a member of the Annelida phylum, which includes segmented worms such as the ragworm and our soon-to-be-friend, the leech. We'll be working with earthworms in the Lumbricidae family.

Earthworms are laid out with a mouth at one end, and an anus at the other. In the middle is the clitellum, a reproductive gland that can secrete a material that helps them form a cocoon. The clitellum is only visible in older worms that are ready to reproduce.

Earthworms have a closed nervous system, with one main blood vessel running down the length of their body. They breathe through their skin, so it is important to keep it moist. They crawl using muscles that are located just underneath the epidermis.



TOP (DORSAL) SIDE OF WORM DARKER THAN BOTTOM (VENTRAL)

Earthworm nervous system

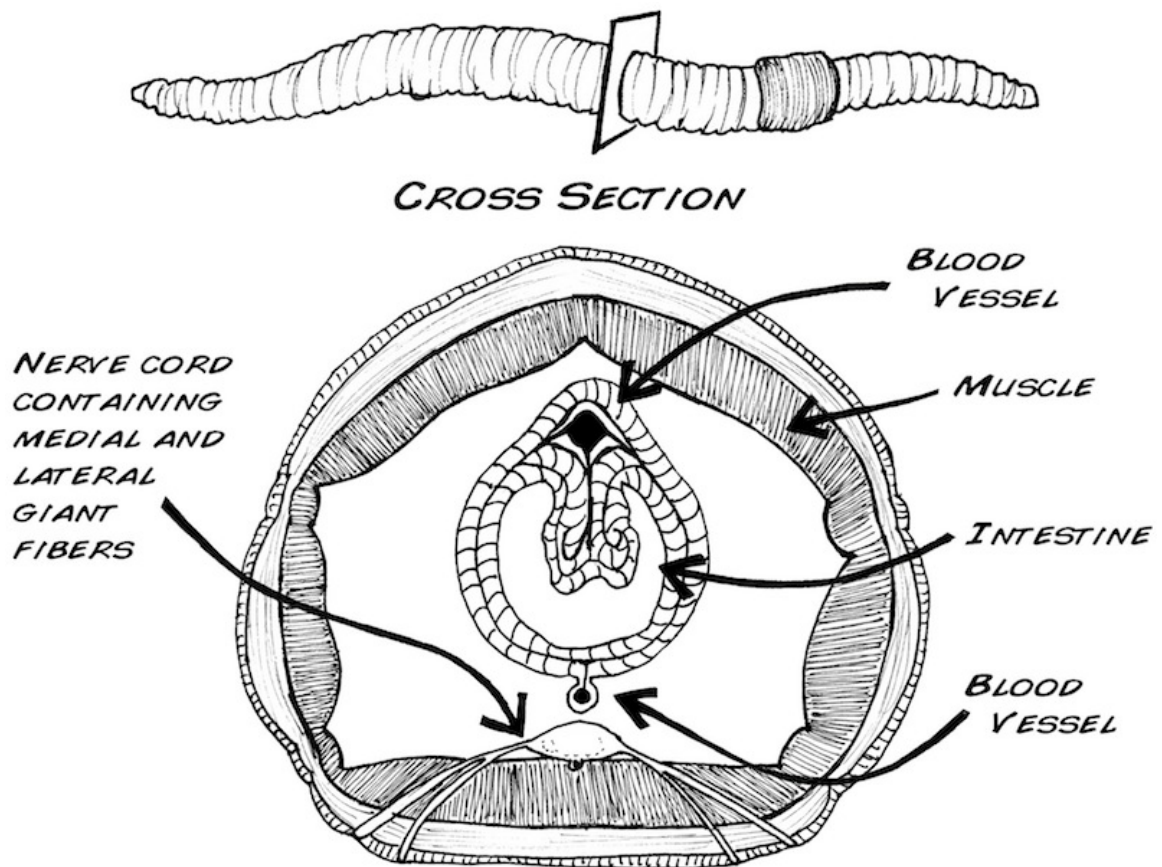
Earthworms have a pretty simple brain: it's actually a fused pair of nerve ganglia, mostly located in their third segment. The earthworm has one main nerve cord on its ventral (bottom) side with three fibers running through it: one medial giant fiber, and two lateral giant fibers.

These fibers are big bundles of axons. They serve as high-speed motor pathways that enable the worm to send movement signals throughout its body for quick, stereotyped reflexes.

Each giant fiber is made up of large individual cells (one per segment) that are electrically coupled to each other through gap junctions. This results in the rapid conduction of action potentials from cell to cell so that each giant fiber behaves as though it were a single axon. There are cross-connections between the medial and lateral fibers, so they often function as one unit.

These fibers differ in important ways. The medial giant fiber transmits information from skin cells in the anterior (mouth) end of the worm, and the lateral giant fibers transmit information from the skin cells of the posterior end of the worm.

It's quite easy to listen in to the activity of these large fibers, even from the skin of the earthworm. Importantly, these fibers fire action potentials very rarely. This type of nervous system activity is often called sparse coding. That means that we should be able to elicit one action potential and cleanly record it.



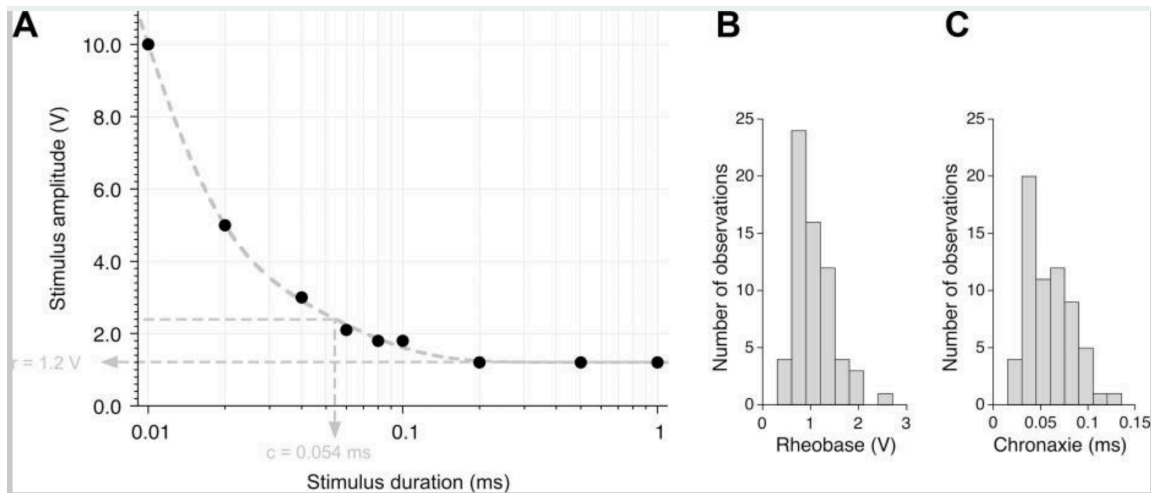
Images from [Backyard Brains](https://www.backyardbrains.com/), © 2009–2026, used under a Creative Commons Attribution-ShareAlike license.

Properties of action potentials

In this lab, we'll determine the **threshold**, **rheobase & chronaxie**, **refractory period**, and **conduction velocity** for the earthworm giant fiber system. We're recording compound action potentials (CAPs) from many axons in these fiber bundles, but the underlying principles are the same.

The **threshold** is defined as the minimum stimulus voltage that elicits an action potential 50% of the time. We'll determine this with short stimuli. Stimulus voltages that don't elicit an action potential are called subthreshold; stimulus voltages that do elicit an action potential are called suprathreshold.

In some neurons, we can elicit an action potential with a lower stimulus voltage if the stimulus length is longer. A longer stimulus length enables charge to build up that can ultimately elicit an action potential. However, there is a minimum stimulus voltage needed, even at infinite stimulus durations. The **rheobase** is defined as the minimum stimulus voltage that can elicit an action potential with very long durations of the stimulus. The stimulus duration at 2× the rheobase is called the **chronaxie**. Chronaxie is a useful measure of the excitability of a nerve — the most excitable nerves have the smallest chronaxie. We can plot this data in a strength duration curve:



The **absolute refractory period** is the minimum amount of time needed for a neuron to fire a second action potential. This is caused by the inactivation of sodium channels after an action potential — it takes time for them to close, to then be reopened by a depolarizing stimulus. Since we're recording from many axons, your measured refractory period will be more variable than for a single axon. We'll define the absolute refractory period as the time when the amplitude of the second CAP is 30% of the first.

After an action potential, the membrane of the axon is also hyperpolarized, due to the slowness of K^+ channels closing. So, there is a period of time where the neuron requires more voltage to fire an action potential. This period is called the **relative refractory period**. In this lab, we'll identify it as the period where the amplitude of the second CAP is 30–90% of the first.

In these experiments you will:

- Stimulate the ventral nerve cord of the earthworm to elicit an action potential
- Determine the voltage that is needed to recruit the lateral giant fibers
- Estimate the absolute and relative refractory period between action potentials
- Calculate the speed of an action potential
- Propose your own hypothesis about conduction velocity
- Conduct your experiment to test your hypothesis

References

- Bähring, R. & Bauer C.K. (2014) Easy method to examine single nerve fiber excitability and conduction parameters using intact nonanesthetized earthworms. *Advances in Physiology Education* 38(3): 253–264.
- Bullock, T.H. (1945) Functional organization of the giant fibre system of *Lumbricus*. *J Neurophysiol* 8:54–71.
- Drewes CD, Landa KB, McFall JL (1978) Giant nerve fibre activity in intact, freely moving earthworms. *J Exp Biol* 72:217–227.
- Kladt et al. (2010) Teaching basic neurophysiology in intact earthworms. *JUNE* 9(1):20–35.
- Rudell & Fox (1991) The Propagation potential: An Axonal response with implications for scalp-recorded EEG. *Biophysical Journal* 60: 556–567.
- Shannon et al. (2014) Portable conduction velocity experiments using earthworms for the college and high school neuroscience teaching laboratory. *Advances in Physiology Education* 38(1): 62–70.

Lab Protocol

There are multiple days dedicated to these earthworm experiments. You are encouraged to complete as many of the experiments below while you have a functioning earthworm.

Supplies	Cables & connectors	Solutions
Earthworm	BNC to single banana adapter (2)	Earthworm saline
Dissection tray	Banana plug cords (2)	Earthworm anesthetic (10% ethanol in Ringers)
PowerLab 26T, connected to computer	Alligator clip adapters (2)	
BioAmp	Needle electrodes	
Sewing pins (3)		
Ruler		
Thermometer		
Faraday Cage		
Large Petri Dish		
Eyedropper		

I. Preparation

Setting up the PowerLab 26T

Our first step is making sure our PowerLab & Amplifier are wired up correctly.

Make sure the PowerLab is turned off.

1. **Connect a single BNC adapter to the (+) Output** of the PowerLab. Plug a red banana plug cable into the adapter.
2. **Connect a single BNC to banana adapter to the (–) Output.** Plug a black banana plug cable into the adapter.
3. Make sure that the BioAmp is connected to the PowerLab, with three different needle electrodes plugged into it.
4. Once you're confident everything is connected correctly, turn on the PowerLab.

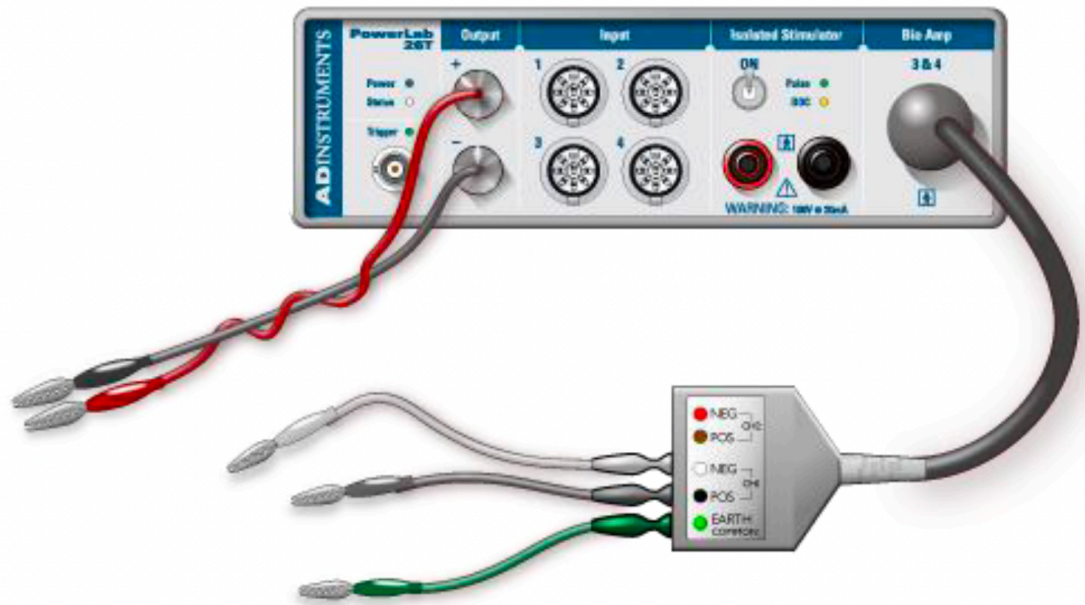


Image courtesy of ADInstruments ©

Setting up LabChart 8

1. **Open LabChart 8 with the settings file you saved during the string experiment.**

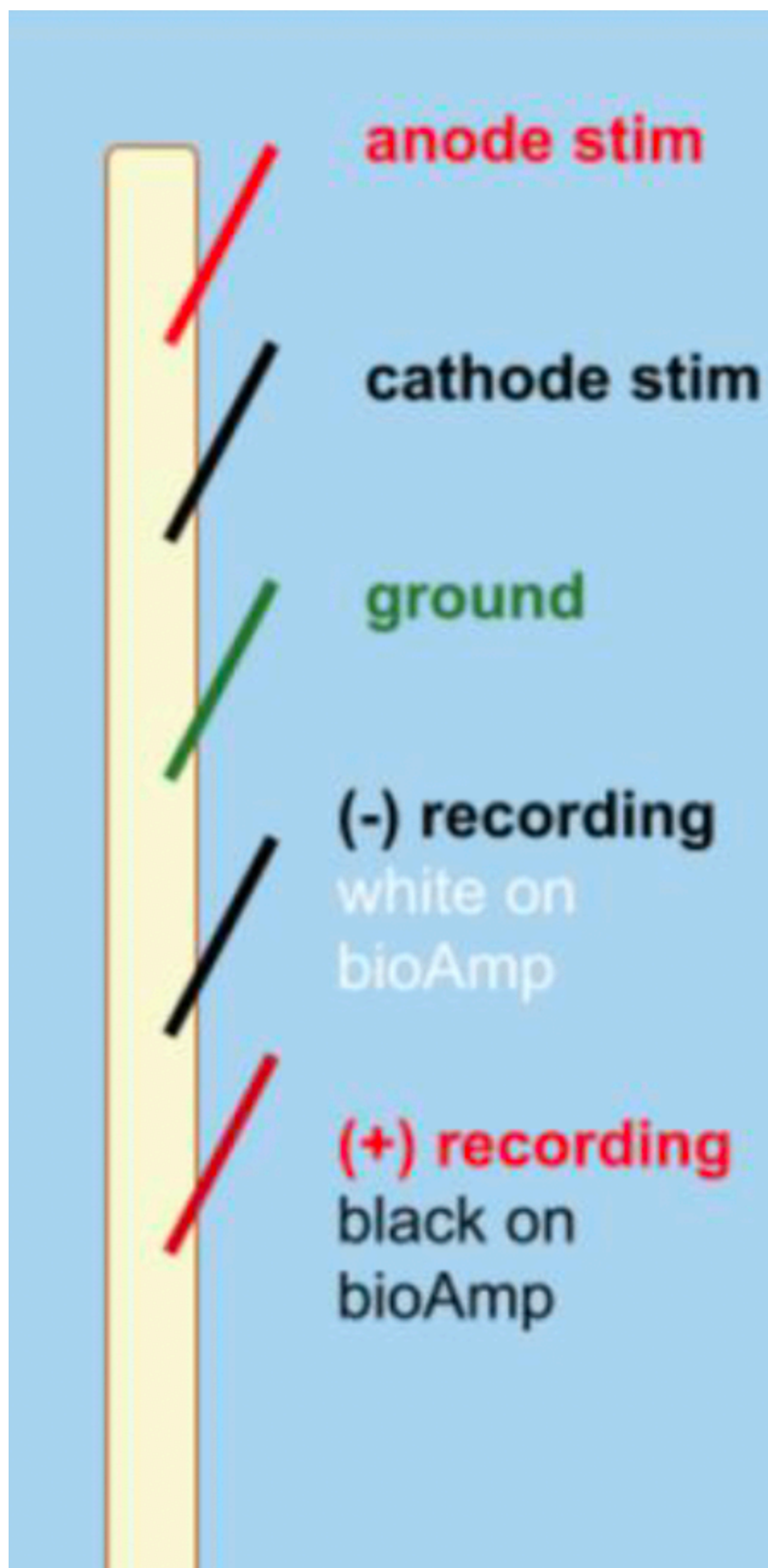
Most settings are saved within this settings file. However, it's always good to ensure that you have the correct settings before beginning.

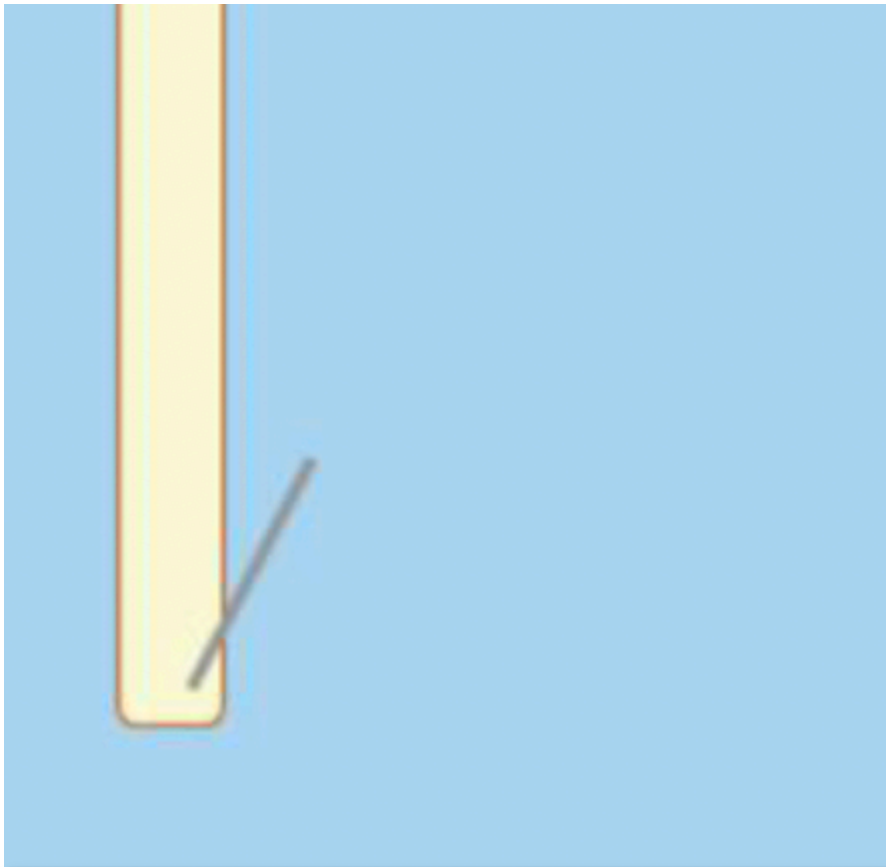
Note: If for some reason you cannot access your settings file, you can find one at <http://bit.ly/labchart>. Use the "Earthworm bioAmp Settings."

2. **Go to Setup > Channel Settings** and confirm that you have three recording channels, and that the third ("BioAmp Recording") is connected to the BioAmp.
3. **Go to Setup > Stimulator** and change the Stimulator settings as follows. Make sure your Marker Channel is Channel 1.

Parameter	Value
Start Delay	0.1 ms
Repeats	1
Max Repeat Rate	1 Hz
Pulse Height	0.2 V
Pulse Width	0.2 ms

Setting up your earthworm





⚠ Don't stretch your worm!

Throughout every step of this procedure, be careful not to stretch the earthworm. This will damage the nerve and your experiment will not work.

1. Obtain a large, lively earthworm. Rinse it with saline if it is dirty.
2. **Anesthetize the earthworm** by placing it in the petri dish with 10% ethanol (in Earthworm Ringer's solution) for 5 minutes.

Note: Do not leave your earthworm unattended in anesthesia. With too little anesthesia, the earthworm will move around during the experiment, and the resulting muscle electrical activity (electromyogram) will drown out the small neural electrical signals you are interested in. Too much anesthesia and the nerves will not fire.

Note #2: Your earthworm may still react to pressure after 5 minutes — you can proceed with the experiment as long as you are able to handle the worm and pin it down.

3. **When your earthworm is anesthetized enough to pin it down**, remove the earthworm from the ethanol and place it on the dissection tray with the dorsal (darker side) up.
4. Pin the earthworm to the tray using one sewing pin on each end of the worm.
5. Place the ruler down next to your worm.
6. **Insert a third sewing pin into the anterior end of the earthworm**, ~1 cm from the first pin. Connect the anode & cathode stimulating electrodes to the first two pins.

Note: The anterior end is the end nearest the clitellum, a smooth, non-segmented band.

7. To keep the worm anesthetized, apply a few drops of the earthworm anesthetic along the length of the earthworm with the eyedropper. Soak up any excess fluid.

Note: Make sure there isn't a large puddle of solution — too much may interfere with your recording.

8. Insert the three recording needle electrodes into the worm (same configuration as the string experiment!).

Note: Make sure no wire is touching any other wire. Be careful when positioning the wires to avoid damaging the worm.

Note #2: It helps to place your recording electrodes as lateral as possible, while still holding down the worm.

II. Determining the threshold of your fibers

In this experiment, you will send a stimulus into the earthworm while recording from it to determine the threshold voltage of the medial and lateral giant fibers.

Determine the threshold to activate the Medial Giant Fiber

1. **Open your Input Amplifier (BioAmp)** for your Action Potential channel to make sure that your signal looks reasonable. You shouldn't see 60 Hz noise, but it should look like a physiological signal. (If you have noise, see Troubleshooting.)
2. **Open the Stimulator Panel** by going to the Setup menu. Your stimulator should start with:
 - Max Repeat Rate: 1 Hz
 - Pulse Height: 0.2 V
 - Pulse Width: 0.2 ms

Note: Leave the Stimulator Panel open — we'll need it throughout the experiment.

3. **Press > Start.** LabChart should be in Scope View and display one 20 ms sweep every time you click Start. For each 20 ms sweep, a stimulus pulse is generated through the PowerLab Stimulator outputs.
4. **Examine the recording** — you should see a quick deflection very shortly after your stimulus pulse. The deflection just after the start of the sweep is your stimulus artifact, caused by spread of the stimulus voltage to the recording electrodes.

Note: The stimulus artifact can be very large in amplitude, and can exceed the range of the recording amplifier. It is okay if it is cut off.

5. **Change the Pulse Height to 1 V.**

- Each time you change the stimulus, click Start and a new page will appear to the left of the Scope View Window. This is the Page Explorer. **Label the individual pages** of your recording as you go along.
- Increase the Pulse Height in 0.1 V steps** until you see an inflection after the stimulus artifact. This means you've successfully elicited a response from the medial giant fiber! Note the stimulus voltage as an estimate of the threshold stimulus strength.

Note: If you can't see anything, right-click on the channel to change the scaling (or use the – and + buttons on the bottom left of the channel).

Note #2: If no response is seen with a stimulus of 3.5 V or greater, contact your instructor for assistance.

- Find a more accurate threshold** by first reducing the stimulus by 0.2 V (or more) to diminish your response. Then, increase the stimulus by 0.05 V between each sweep. You should report a threshold (Table 1) to the hundredth decimal point (e.g. 1.25 V).
- Save your data, but do not close the file.

Determine the threshold to recruit the Lateral Giant Fibers

In the next stage, you will attempt to elicit an action potential in the lateral giant fibers.

Note: Remember to keep the earthworm anesthetized!

- Set the Pulse Height to your threshold for the MGF.
- Keep increasing the stimulus in 0.1 V steps until you see a second inflection in your recorded trace.
- Once you see a second inflection, follow the same procedure as above to determine the threshold where you can elicit an action potential from the lateral giant fiber about 50% of the time. Record the stimulus threshold in Table 1 below.
- Save your data, but do not close the file.

Note: In some worms, the median and lateral fiber responses may not be separated — they will appear to have the same threshold. Other worms may not show the second response, possibly because the lateral giant fibers are damaged.

- Save your LabChart file.

Table 1. MGF and LGF compound action potential (CAP) properties.

	Threshold	Amplitude	Latency
MGF			
LGF			



Questions for reflection

- Did the shape or amplitude of the CAPs change with more stimulation voltage? What does this tell us about the action potential that is firing?
- Which fiber fires an action potential sooner (with a shorter latency)?
- Why would the MGF and LGF latencies differ?

II. Calculate the rheobase

In Part III, we'll change the stimulus amplitude and duration to calculate the rheobase and chronaxie of the earthworm medial giant fiber (read the Background if you forget what these are).

In the previous experiment, we gave a stimulus for 0.2 ms. Now, we'll modify the stimulus duration and amplitude to determine the minimum stimulus requirement, regardless of duration.

1. Write your MGF threshold in the column for 0.20 ms stimulus duration in Table 2.
2. Change your Pulse Height to the threshold for your MGF.
3. Systematically change the duration of your stimulus (Pulse Width) to the values in the table below. As necessary, change the stimulus voltage to successfully get a CAP from your MGF.
4. For each duration, record the stimulus voltage (Pulse Height) values that successfully elicit a CAP in Table 2.

Note: Be sure to label your pages in Scope view accordingly. You don't need to save the pages that do not elicit an action potential.

Table 2. Data for strength-duration curve.

Duration (ms)	0.06	0.08	0.10	0.20	0.40	0.60	1.00	1.40	1.80
Stimulus Amplitude (V)									

IV. Determine the refractory period

In the next step, you'll determine the shortest time between stimuli where you can elicit two different CAPs from the MGF. In other words, we'll determine the relative and absolute refractory period of these fibers.

1. Set your pulse height so that you can comfortably elicit a response from the MGF.
2. **In Setup > Stimulator, change Repeats to 2** in order to play multiple pulses.
3. We also need to change the time between each stimulus. PowerLab thinks of this interval as Hz. To start, we want 15 ms between each stimulus.

Note: Remember that Hertz (Hz) means events per second. So, 1 Hz = 1 event per second, and 10 Hz = 10 events per second. $1 \div \text{frequency (in Hz)} = \text{time between}$

stimuli (in seconds).



Questions for reflection

- How many milliseconds would there be between each stimulus at 10 Hz stimulation?
- For 15 ms between each stimulus, what should the frequency of stimulation (in Hz) be?

4. Fill out this chart with the corresponding Hz values for each interval length:

Interval (ms)	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1
Freq. (Hz)															

5. **Change your Max Repeat Rate** to the equivalent frequency for 15 ms intervals.
6. Change your Pulse Width to 0.2 ms and your Pulse Height to the threshold for your MGF.
7. **Press > Start** to record a trial with two stimuli, separated by 15 ms.
8. Right-click and choose **Add Marker > Clamp to Trace** to place a marker at the moment the stimulus occurs. Move your cursor to the peak of the action potential. Look at the ΔV on the right-hand side to see the change in voltage from that point.
9. **Measure the amplitude of your first and second action potentials.** Determine how strong the second was relative to the first (e.g. if AP #2 amplitude was half as high as #1, it would be 50%). Record your response in row one of Table 3.
10. Decrease the duration between stimuli by 1 ms by changing the Repeat Rate to the Hz for a 14 ms interval.
11. **Record the amplitudes and corresponding percentages for each stimulus interval in Table 3.**
12. At some point, AP #2 will diminish below 30% of AP #1. We can consider this our absolute refractory period.

Note: Depending on the latency of your MGF CAP, it may be difficult to see a CAP with a stimulus interval of 1 or 2 ms. Reduce the stimulus interval to as short as you can before the MGF CAP is cut off by the second stimulus. If the shortest stimulus interval is >3 ms, you should move your pins closer together.

13. Save your LabChart file.

Note: If you have data that doesn't seem quite right, you should repeat your experiment. You must be upfront about your number of repeats in your lab report, comprehensively reporting all of the data that you collected.

Table 3. Amplitudes of CAPs elicited by stimuli with decreasing stimulus intervals.

Stimulus Interval (ms)	#1 CAP Amplitude	#2 CAP Amplitude	% #2/#1
15			
14			
13			
12			
11			
10			
9			
8			
7			
6			
5			
4			
3			
2			
1			



Questions for reflection

- What was the shortest duration between stimuli where the second action potential was less than 90% but more than 30% of the original CAP amplitude?
- What was the shortest duration between stimuli where the second CAP was diminished below 30%?
- Which of these values is your absolute refractory period, and which is the relative refractory period?

V. Compute the conduction velocity

(If you're doing this on a different day, follow the same steps for preparing LabChart & your earthworm as you did in the previous earthworm experiment.)

How far is your first recording pin (–) from your cathode (–) stimulation pin? Be sure to include units: _____

1. In **PowerLab**, press **Start**. Confirm that you have two recording channels.
2. Follow the same protocol as in the previous experiments to determine the threshold required to elicit an action potential from your medial giant fiber.

What is the stimulus voltage needed to elicit an MGF action potential from this worm?

Determine the MGF latency to spike

1. Using the Marker tool, measure the latency (in seconds) from the peak of the stimulus artifact to your elicited action potential.
2. Repeat this **10** times and record your data in Table 4.

Table 4. MGF conduction velocity data at room temperature.

Trial	MGF Cold Temperature Latency (s)	Speed (m/s)
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		

3. Since we know when the AP occurred and the distance between your electrodes, we can calculate the speed of the action potential. Record these values in the "Speed" column of the table above. (*Reminder: speed = distance/time.*)

VI. Design your own experiment to test the impact of temperature on MGF conduction velocity

For this final part, you will design your own experiment to test the impact of temperature on conduction velocity. We will provide ice and a thermometer, but the rest is up to you! If you are performing this experiment on a separate worm, be sure to measure the threshold for the MGF as described above.

Work with your group to determine a protocol for running your experiment — you'll need to include this in the Methods section of your lab report. Think about what someone needs to

know in order to replicate this experiment exactly as you did.

Table 5. MGF conduction velocity data.

Trial	MGF Cold Temperature Latency (s)	Speed (m/s)
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		

Conduct your experiment

Work with your group to determine a protocol for running your experiment — you’ll need to include this in the Methods section of your lab report.

Record your results in the MGF Experimental AP Latency column of Table 4. You’ll need these later for the Results section of your lab report.

Troubleshooting

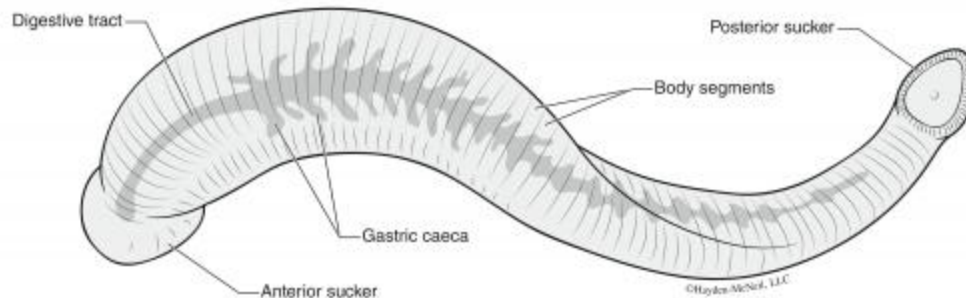
Observation	Likely issue	Possible solution
The recording trace is flat	Something is disconnected	Check all of your connections
I can't see a stimulus trace in the Stimulus channel	You don't have a marker channel set up	Go to Setup > Stimulator to set up the Marker Channel
I can't see a stimulus artifact in my action potential channel	Your electrodes may not be in the worm, or stimulus amplitude is too weak	Ensure all electrodes are in the worm (but not too medial), and try stimulating at 0.5 V
My worm is bleeding	The electrode is through the blood vessel	Move the electrode more lateral; if you don't get an AP after this, get a new worm
There is a stimulus artifact, but no visible response to stimuli between 1–3 V	Your electrodes may not be placed well, or your worm may be damaged	Try to reposition the electrodes on a different place in the worm. If this doesn't work, get a new worm.
My recording just looks like a big up and down squiggle (like a sine wave)	You're picking up 60 Hz noise	Ground everything (Faraday cage, ground recording pin, amplifier) to the ground pin on the back of the PowerLab. Averaging multiple trials can also get rid of noise.

Leech Electrophysiology

Background & Goals

Leeches as a model for neuroscience

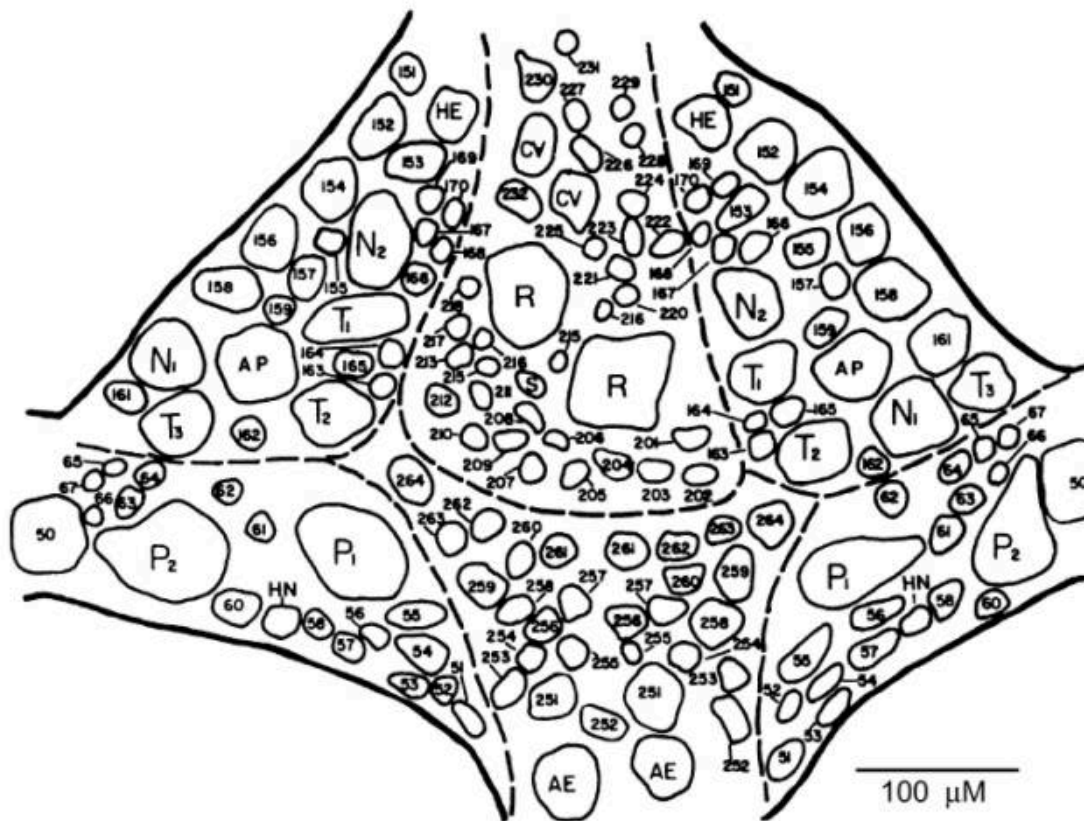
It might seem strange, but leeches are fantastic models for neuroscience experiments. They have a rich, well-defined repertoire of behaviors and an easily accessible, stereotyped nervous system. Over the next few labs, we'll record from the medicinal leech, *Hirudo medicinalis*, to determine some physiological and morphological properties of neurons.



Leech nervous systems

Leeches are annelids (segmented worms), just like our earthworm friends. They have a ventral nerve cord containing a total of 32 ganglia: 4 fused head ganglia, 21 segmental body ganglia, and 7 fused tail ganglia. We'll be recording from the body ganglia.

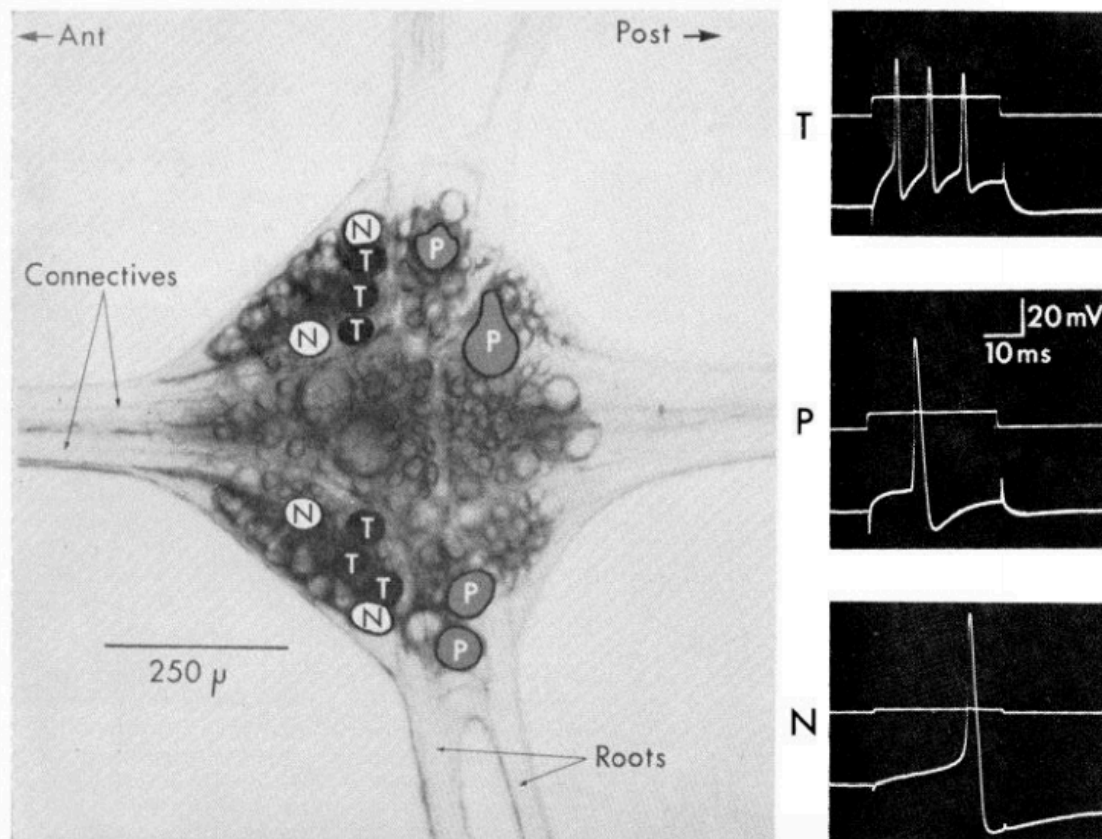
Each ganglion has about 400 neurons in it, most of which are paired — in other words, most of them are connected to one other neuron. The ganglia are all connected via connectives that run the length of the leech. In addition, each segmental ganglion has lateral roots that reach out to the skin. The neurons in each ganglion are laid out in stereotypical positions, and have well-defined electrophysiological properties. We'll be looking at the ganglion from the ventral side, where many neurons are accessible:



Adapted from Macagno (1980), "Number and Distribution of Neurons in Leech Segmental Ganglia."

Types of cells in the leech

- **Rz (Retzius):** the biggest cells of the leech ganglion, located in the middle of the ventral side of the ganglion. These cells often show a second component in their action potential due to the close electrical coupling with the other Rz cell. These cells do not overshoot.
- **Sensory neurons:** Each of these neurons projects its axon from the ganglion to a particular territory of the skin, where they have specialized mechanoreceptors. Unlike the Rz cell, these sensory neurons have overshooting, very high amplitude action potentials.
 - **T (touch):** three of these in the anterolateral packets on each side of the ganglion. Typically a very short, relatively low amplitude waveform (compared to other sensory neurons).
 - **P (pressure):** two of these in each posterolateral packet.
 - **N (nociception):** two in each of the anterolateral packets. These cells have very long undershoots.
- **AP (anterior pagoda):** large cells near the center of the anterolateral packets.
- **AE (annulus erector):** motor neurons to the AE muscles in the body wall; medium-sized cells in the posterior ventromedial packet.
- **S:** an unpaired cell, unlike the cell types listed above, much smaller than the other neurons listed. Record from this neuron and you shall gain infinite respect from the Gods of leech neurobiology.



Adapted from Nicholls & Baylor (1968), "Specific modalities and receptive fields of sensory neurons in the CNS of the leech."

About the microelectrode & amplifier

Instead of using a wire, we'll be recording from these cells with a glass pipette that has been pulled to have a very tiny and sharp tip and is filled with conductive fluid. These are typically called microelectrodes. Recording from nervous systems in this way was a huge leap forward in 1949 — before then, we weren't able to record from inside of cells (Ling & Gerard, 1949).

Back in the day, physiologists would make these small pipettes by pulling the glass over a flame. Now, we've got a dedicated machine to pull pipettes, allowing us to reproduce many microelectrodes that are the same size.

And about the size — these micropipettes are really small. This small cross-sectional area of the microelectrode greatly reduces the flow of current through the electrode. In other words, it has a really high resistance (often imprecisely termed "impedance" in electrophysiology circuits). This is the quality that allows us to record very tiny changes in voltage in our cells. This high resistance is necessary, but also brings up some other problems. With really high resistance across our electrode, membrane, and PowerLab, we would almost record no voltage from a 60 mV cell.

So, for these labs, we're using a different amplifier. Because the impedance of our microelectrodes is much higher than the electrodes we used for the earthworm, we need an amplifier that can "impedance match" the microelectrode with the PowerLab. It also means that the intracellular recording system must be shielded from electrical noise much more than the extracellular system.

But with all this in place, you'll get a chance to see some real action potentials from a strange, slightly terrifying creature.

In these labs you will:

- Learn how to operate and calibrate an intracellular amplifier
- Record action potentials from Retzius cells, both with and without current injection
- Fill a single cell in the leech with a fluorescent dye to visualize its structure

What's coming up

This module spans **three lab days**:

- [Intracellular Lab](#) — troubleshoot noise, check electrode resistance, and balance the bridge on the intracellular amplifier.
- [Retzius Recording](#) — find and record from a Retzius cell, both spontaneously and with current injection.
- [Filling the Retzius Cell](#) — fill a leech neuron with a fluorescent dye and image it under the microscope.

Resources

- [HHMI Virtual Leech Lab](#)
- [JOVE Video: Intracellular Recording & Sensory Field Mapping](#)

References

- Macagno, E.R. (1980) Number and distribution of neurons in leech segmental ganglia. *Journal of Comparative Neurology* 190(2): 283–302.
- Nicholls, J.G. & Baylor, D.A. (1968) Specific modalities and receptive fields of sensory neurons in CNS of the leech. *Journal of Neurophysiology* 31(5): 740–756.
- Ling, G. & Gerard, R.W. (1949) The normal membrane potential of frog sartorius fibers. *Journal of Cellular and Comparative Physiology* 34(3): 383–396.

Intracellular Lab

The goal of this first experiment is to troubleshoot the noise in your rig, learn how to check the electrode resistance, and balance the bridge on your amplifier. You'll also get familiar with the microscope we're using and finding tiny things in a dish.

In subsequent labs, we'll be recording from tiny cells in the leech ganglia, as well as filling these cells with a fluorescent dye. Those experiments will require that you can quickly identify and manipulate a microelectrode underneath the microscope in order to find your cell of choice. Time is of the essence here — we're keeping cells alive in a dish, and they won't last forever. Becoming proficient at these steps will save you time during the recording experiments, and therefore increase your likelihood of success.

Supplies	Cables & connectors	Solutions
Clear plastic holder for petri dish	DIN8 plug to BNC Cable (MLAC25)	3M KCl
PowerLab 26T, connected to computer	BNC Cable	Saline
A-M Systems 3100 Intracellular amplifier & silver head stage probe	Banana plug cords (5)	
Microelectrode holder (AM Systems #675440)	Alligator clip adapters (5)	
Microelectrode (BF100-58-10)	Ground wire & clay or wax	
Faraday Cage		
Metal baseplate & foam pads		
Micromanipulator & stand		
Stereomicroscope on boom stand		
Light source		
Syringe & needle		
4 cm diameter petri dish with saline		
4 cm diameter petri dish with microbeads		

I. Tiny beads, optics, & manipulators

This first activity will prepare you for recording from a leech ganglion. The steps you take to set up your microscope, light source, and manipulator will be almost identical next week. You might have already used a microscope before, but it's less likely that you've had to do something very precise while doing so!

Determining the diameter of your viewing field

1. Compute the total magnification of your microscope at 1.8X by multiplying 1.8X times the objective magnification (typically 10x, check your oculars). Write this value in Table 1.
2. Similarly, compute the total magnification for ~5X and 11X. Write these values in Table 1.
3. Place the ruler underneath the microscope at 1.8X and look through the oculars. Determine the diameter of your viewing field and record this in Table 1.
4. It'll be tough to do the same at high magnification, so instead we can use this formula:

high power field diameter = (low power field diameter × low power magnification) ÷ high power magnification

5. Using this equation, determine the diameter of the viewing field for 5x and 11x.

Table 1.

Objective	Total Magnification	Diameter of viewing field (mm)
1.8x		
5x		
11x		

Setting up your lights and microscope

1. First, set your dish with microbeads into the clear acrylic holder. Be careful placing these into the holder — it's designed to fit on the ledge inside.
2. Arrange your lights so that they're shining onto the dish.
3. Bring the beads into focus at a low magnification (1.8–3x).
4. Move to the highest magnification, and bring the beads into focus. Adjust your light source so that the beads are clearly visible.
5. Use your calculated viewing field diameter to estimate the size of the smallest bead in your dish.

Given the diameter of your viewing field, what is your estimated diameter of the smallest microbead in your dish? _____

Using your manipulator

1. Get a microelectrode from the front and use the syringe to fill it with KCl.
2. Gently put the microelectrode onto the wire of the microelectrode holder and screw it in.
3. Put the micropipette holder into the silver headstage by pushing the gold pin into the bottom.
4. Make sure that your manipulator arm is at a right angle. Any angles that are not perpendicular will make it difficult to maneuver your micropipette.
5. Move the manipulator base (you need to turn the magnetic switch to OFF) close to the dish so that the tip of the "electrode" is within 3 inches of your dish.
6. Use the knobs on the manipulator to bring the tip of your pipette about ½ inch above your beads.
7. Bring the focus above the dish so that the tip of the pipette is now in focus.
8. Lower the focus, and bring the tip of the pipette into that field of view.

9. Repeat this until the tip of the pipette is at the same focus as the glass beads.
10. At a high magnification (11X), you should see the glass beads being repelled by the tip of the pipette, or be able to push them around.
11. Once you have the tip of your pipette at the same focus as your beads, raise your electrode and remove the dish of beads.

Troubleshooting

Observation	Likely issue(s)	Possible solution
The view through the scope is blurry	You're out of focus	Adjust the focus
There are weird light artifacts and reflections that make it difficult to see	The light is not set up well	Change the angle, intensity, and distance of the light source. Try 1 light source vs. 2.

II. Preparing your recording setup

Setting up PowerLab & the Amplifier

1. Make sure that your PowerLab is off.
2. Configure your PowerLab & amplifier as in Figure 1:

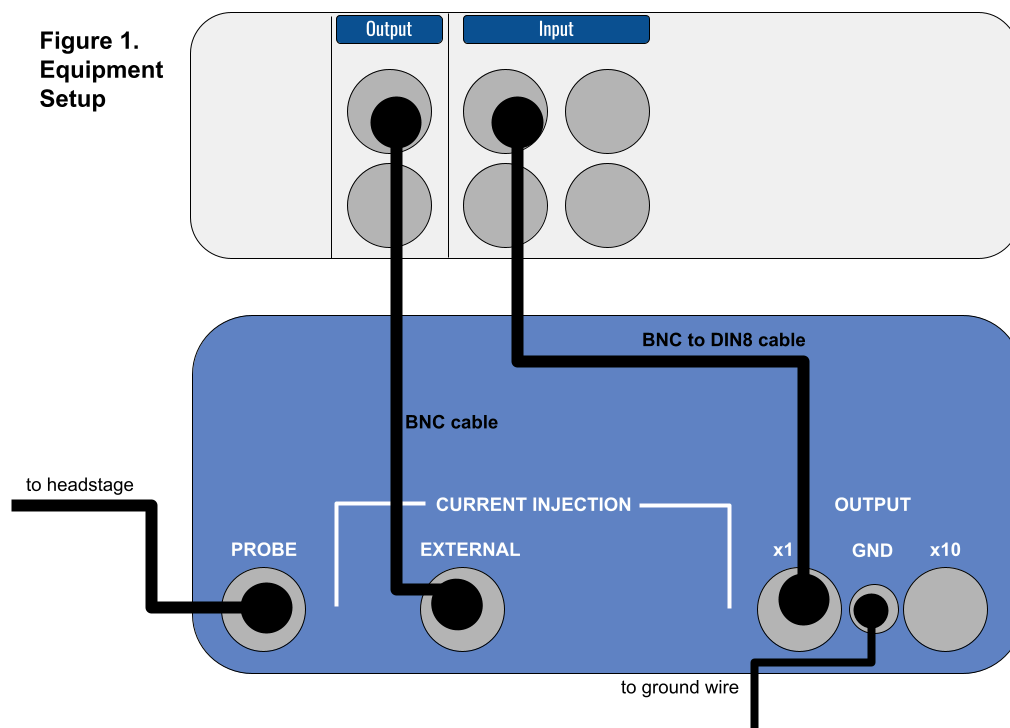


Figure 1. Equipment setup.

The amplifier settings should be as follows:

Setting	Value
DC Offset knob	Counterclockwise
DC Offset (+/Off/-)	Off
Transients	Counterclockwise
DC Balance	Counterclockwise
Low Pass	5 kHz
Ω TEST	Off
Notch	Off
Capacity Compensation Knobs	Fully counterclockwise
Current Injection	Fully counterclockwise (zero) — VERY IMPORTANT!
Current Injection Knob (Cont/Off/Momen)	Off

3. Once everything is connected, turn on your PowerLab and the Amplifier.

Note: The AM 3100 amplifier needs to warm up for 5 minutes before doing any recording. You can proceed to set up your electrode though.

Prepare the saline, manipulator, electrode, and ground wire

- Your manipulator and electrode should already be set up from Step I. However, if you broke your electrode during Step I, you should prepare a new one.
- Fill a small petri dish with saline and put it into the holder.
- Using the clay, position the ground wire so that it is in the saline but at the edge of your petri dish. Your ground wire should be plugged into "GND" on the amplifier. You may need to piggy-back it onto another banana plug cord to do so.

III. Noise, resistance, and balancing the bridge

Check for noise in your signal

First things first — with this recording configuration, we'll probably pick up much more noise than we need to troubleshoot.

1. Maneuver the tip of the microelectrode into the saline.

Note: Be very careful while moving your microelectrode around! If you break the tip of it, you'll need to get a new electrode.

2. Open LabChart with the Leech Settings file (<http://bit.ly/labchart>).

3. Open the Input Amplifier for your recording ("Action Potential") channel.

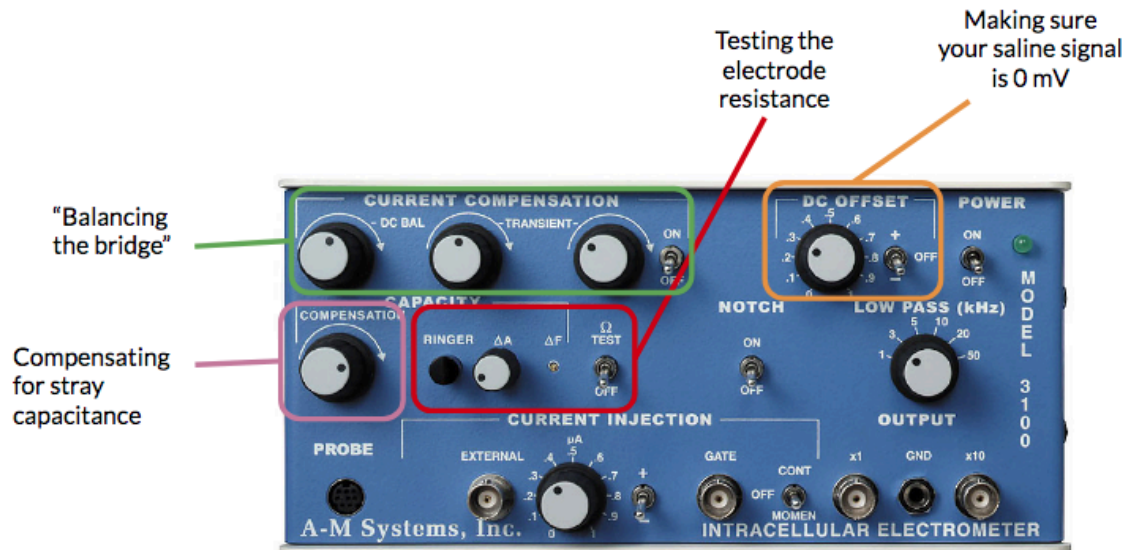
4. Change the zoom on your signal and the range (if necessary) so that your signal fills about $\frac{2}{3}$ of the screen.

5. Chances are you'll see 60 Hz noise. In order to get rid of this, ground (one-by-one, watching to see how they change the signal) your: Faraday cage, metal plate, microscope, and light (clip on to the metal parts) to the ground pin on the back of the PowerLab. It will help to piggy-back banana plug cords.

6. Once you've reduced your noise as much as possible, you can move on.

Note: As you're moving things around your rig, make sure that your lights can reach your petri dish and that you can easily manipulate the electrode & knobs on your amplifier.

For the next steps, you'll need to use these knobs on your amplifier:



Check the resistance of your electrode

1. With the Input Amplifier window open, note the voltage of the trace. It will likely not be exactly at zero. This is due to the slight differences in the potentials of the ground and recording electrodes, and will change when you change electrodes.
2. Switch the DC OFFSET to positive (+). Using the DC OFFSET knob on the amplifier, position the tracing so that it is very close to 0 volts. You may need to set the switch next to DC OFFSET to negative (-) depending on what direction you need to move your signal.



Note: If the voltage is drifting wildly, make sure that your ground wire is in place in the saline and that everything is still grounded properly. Restarting LabChart & turning on/off the PowerLab is also a good troubleshooting step.

3. With the tip of your electrode in the saline, press Start in LabChart. You should be viewing in Chart mode.
4. Very quickly, turn on and off the Ω TEST on the Amplifier. This Ω TEST sends 10 nA of current into the electrode.
5. Press Stop in LabChart.
6. For the data you just recorded, autoscale the channel in time and voltage so that you can clearly see a square wave pulse.
7. If your waveform is not square, turn the CAPACITY COMPENSATION knob on the amplifier slightly clockwise and repeat the Ω TEST. You can treat LabChart like an oscilloscope and let it run (press Start) while you do these tests.

Note: Do not turn the CAPACITY COMPENSATION knob too far — it will result in uncontrolled oscillations in your output. If you do this while recording from a cell, you will greatly damage the cell.

8. Once you've squared up your waveform, measure the height of this square wave in volts.

Calculate the resistance of your electrode according to Ohm's Law ($V = IR$). In the units we're working with here, $1 \text{ M}\Omega = 1 \text{ mV} / 1 \text{ nA}$. Since the Ω TEST sends 10 nA, with 1X gain a 10 mV shift in the output voltage corresponds to 1 M Ω resistance.

Electrode resistance = _____

The resistance of your electrode should be between 20–60 M Ω . If it's too high (>60 M Ω), it might be clogged. Use the Ringer button and try again. If it's too low (<20 M Ω), the tip of your electrode probably broke — get a new electrode.

Balancing the bridge for current injection

We need to balance the bridge circuit of the amplifier if we want to inject current into a cell. The cell and the electrode each have their own capacitance and resistance. You dial out the resistance of your electrode before you impale the cell; otherwise your recording will reflect the voltage change across the electrode, instead of the actual voltage of the cell.

1. Set the CURRENT INJECTION (CONT-OFF-MOMEN) switch to CONT and the μ A knob rotated fully counterclockwise.

Note: If this switch is not on CONT (continuous), no current will be injected.

2. Go to the Setup menu and select Stimulator. Set up the Stimulator to deliver a 10 mV (1 nA current) as a train of 100 pulses:

Parameter	Value
Start Delay	0 seconds
Repeats	100
Max Repeat Rate	0.66
Pulse Height	0.01 V (10 mV)
Pulse Width	1 s

3. Set Channel 2 as your stimulus Marker Channel.
4. Close the Stimulator dialog and open the Stimulator Panel. Start recording, and if necessary zero the electrode again using the DC OFFSET knob.
5. Turn the stimulator on to inject a train of 1 nA of current pulses.
6. Adjust the DC BALANCE knob as necessary to remove the electrode voltage drop from the output signal.
7. There will still be “rabbit ears” or transients — these are large spikes when you turn the current on and off — but very little change in standing voltage:

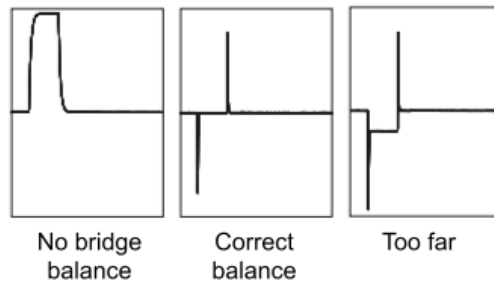


Figure 2

Figure 2. Bridge balance traces.

8. Adjust the Current Compensation TRANSIENT controls to minimize transients occurring when the current is gated on and off. The left knob controls the slope of the transient, the right knob controls the peaks.
9. Stop recording.
10. Set the CURRENT INJECTION knob back to OFF.
11. Save your data as "Leech electrode setup". If you made any changes to the settings file (e.g., Channel names, range, etc.) you'll also want to save it as a new settings file by going to File > Save As LabChart Settings File. Save this in a safe, clouded place.
12. Do not move any of the knobs or cords. You'll need these to be in place for the next lab.

Retzius Recording

Now that you're comfortable with the microscope, manipulator, and amplifier, it's time to record from a real leech ganglion. In this lab, you'll find and impale a Retzius cell, characterize its spontaneous firing, and inject current to see how it responds.

I. Preparing for recording

First, you need to fill and arrange your electrode, troubleshoot noise, and configure your amplifier in the same way that you did in the [Intracellular Lab](#).

Once you've confirmed that your manipulator and recording equipment is ready to go, get a leech ganglion.

1. Maneuver your electrode into the saline bath. At the moment, there isn't any need to get it close to the ganglion — as long as it's in the saline, you can measure the resistance and troubleshoot noise.
2. Make sure the peak-to-peak noise level of your recording is < 5 mV. There shouldn't be any obvious sinusoidal noise in your data. If there is, find the source and ground it.

3. Test the resistance of the electrode (follow the steps from the previous lab):

Electrode resistance: _____

4. Balance the bridge — it shouldn't be too far off since it should be balanced from the previous lab.

Getting your leech & microelectrode in focus

Before we can do any recording, we need to ensure that your light is set up well so that you can see your ganglion.

1. Gently rest your petri dish in the plastic holder.
2. Bring the leech ganglia into focus, and adjust your light and equipment positioning as necessary.
3. When you can clearly see your ganglia, see if you can spot the two Retzius cells in the middle (Figure 3).

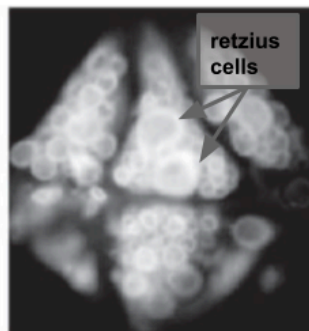


Figure 3. Ventral view of leech ganglion.

Figure 3. Ventral view of leech ganglion.

Note: If you cannot see the two Retzius cells, it might be because your lighting needs to be adjusted, or because your ganglia is mistakenly set with the ventral side up.

4. Using the clay, position the ground wire so that it is in the saline but at the edge of your petri dish. Your ground wire should be plugged into "GND" on the amplifier.

II. Recording from a Retzius cell

Finding and impaling a Retzius cell

1. Slowly and carefully move the tip of your electrode down towards the ganglia, until it is very close to the Retzius cells.
Note: It helps to do this at medium magnification (5–8X) but in the end, you should be looking at your cells and electrode at the highest magnification possible (11X).
Note #2: The tip of your electrode will probably be very hard to see. You can move it slightly from side to side if you lose track of it. You may need to adjust your light source so that you can see both the ganglion and the tip of the electrode.
2. While hovering above the cell but before getting too close, check your resistance again. Be sure to also keep the recording voltage close to zero while you're outside of the cell.
3. Once you think you're as close as possible to the cell, even dimpling the cell under the pressure from the electrode tip, start recording.
4. Firmly tap the electrode to encourage it to enter the cell. While tapping, watch the recording screen.
5. If you entered the cell, you should see your membrane potential rapidly drop to about -50 mV and stay there.
Note: If your membrane potential goes below -65 mV, you're likely in a glia cell.
6. If all goes well, you should see your Retzius cell firing very clear spontaneous action potentials!
7. If your first attempt is unsuccessful, slightly advance the electrode towards the cell and try again. If this doesn't work, reposition the electrode to a different part of the cell and try again. Frequently check the resistance of your electrode to ensure that it is not broken or clogged.

Once you have successfully impaled a Retzius cell, take note of the resting membrane potential of the cell:

Resting membrane potential: _____

Note: The resting membrane potential will change slightly over time. Your most accurate measurement will be from when you first pierce the cell.

Also take note of the shape of this action potential. Does it overshoot (past zero)? Does it undershoot?

Recording spontaneous action potentials ★

We'll collect 10 different 10 second windows of spontaneous action potentials from your cell, in order to estimate the spontaneous firing rate and measure the action potential height, spike width, and after-hyperpolarization of your cell.

Note: You can also do this with the data you recorded directly after you impaled the cell. But having 10 separate windows will make it a bit easier to calculate the spontaneous firing rate.

1. Configure Scope View to collect data with a 10 second duration.
2. Set repeats to 10, and press Start.
3. Confirm that you have 10, 10 second recordings with spontaneous action potentials by looking through your pages.

You will ultimately use this data to fill in Table 1, but don't take the time to do this now. Move on to stimulating your Retzius cell while you have one.

III. Stimulating the Retzius cell ★

Once you have characterized the spontaneous activity of your cell, we can also inject current to see how the cell responds to inputs.

1. Set up your Stimulator as a pulse with the following parameters:

Parameter	Value
Delay	5 ms
Max Repeat Rate	0.5 Hz
Repeats	1
Duration	500 ms
Pulse Height	10 mV (PowerLab will tell the amp to send 1 nA for each 10 mV)

2. Make sure that your CURRENT INJECTION toggles are on positive (+) and CONT.
3. Configure Scope View in LabChart so that you can see one, 1 second sweep when a stimulus pulse is sent.
4. Press Start to send current into your cell. Observe whether or not this elicits action potentials from the cell, and whether the cell adapts to the stimulus.
5. Repeat the stimulation 10 times (or more, if it seems highly variable). You'll need these repeats for your spike train metrics in your lab report.
Note: For this step, do not use averaging. Averaging will repeatedly send the same stimulus with only a short break. Give the cell a few seconds to recover in between stimuli.
6. Repeat the stimulation with the following current levels (with 10 repeats per current level): -0.5 nA, 1.5 nA, 2.0 nA. You'll ultimately need to count the number of action potentials for each of these (you should have four different current levels total) for a figure in your lab report.

Sample data notebook for Retzius cell recording

Table 1. Spontaneous action potential characteristics.

Action Potential	AP Height	AP Spike Width	After-hyperpolarization Amplitude
1			
...			
Mean (or Median)			
SD (or IQR)			

Note: This table is a suggestion for how to ultimately collect data for this portion of the report. Set up a similar table after you have collected your data.

Table 2. Characteristics of spike trains elicited by a 500 ms, 1 nA square wave stimulus.

Trial	Latency to spike	First interspike interval (ISI)
1		
...		
Mean (or Median)		
SD (or IQR)		

Note: This table is a suggestion for how to ultimately collect data for this portion of the report. Set up a similar table after you have collected your data.

Filling the Retzius Cell

In this final leech lab, you'll fill the cell you recorded from with a fluorescent dye so that you can visualize its structure under the microscope.

Additional Supplies

- Glass slide & coverslip
- Glycerol
- Bulb & glass pipette
- Small vial of either Alexa Fluor 488 (green) or Alexa Fluor 594 (red) conjugated dextran dye (5% in dH₂O)
- Phosphate buffer (PB)
- 2% Paraformaldehyde (PFA) in 0.1 M PB
- Waste containers (2)

I. Preparing Your Experiment

First, we have to prepare an electrode that has dye in the tip. These dyes diffuse well through cells and will latch onto proteins when we fix the ganglia.

We'll use one of two dyes: an Alexa Fluor 488 dextran (green), or an Alexa Fluor 594 dextran (red). If you have time today, you are welcome to fill more than one cell to compare two neurons.

1. You'll be given a small vial with dye in it.
2. Place your glass pipette with the back end (the non-sharp end) into the small vial. The dye will be wicked along the fiber inside of the pipette.
3. Hold the pipette there for ~1–3 minutes until you can see traces of dye in the sharp tip of the glass pipette.
4. Remove the pipette from the small vial of dye.
5. Using the syringe, fill the pipette with 3M KCl solution. Leave a small gap (3–5 mm) of air between the dye and the KCl.
6. Carefully attach your glass pipette into the gold microelectrode holder by sliding the metal wire into the pipette, pressing firmly, and screwing it tight. Make sure that the pipette is not going to fall out of the holder.
7. Attach the pipette holder to the head stage.

Checking the resistance of your electrode

Note: For this experiment, you need to check the resistance, but you do not need to balance the bridge.

Measure the resistance of your electrode by following the steps in the previous lab protocol. In the same way, use Ohm's law to calculate the electrode resistance. The resistance with the dye will be higher than before, but should still not be higher than 100 M Ω .

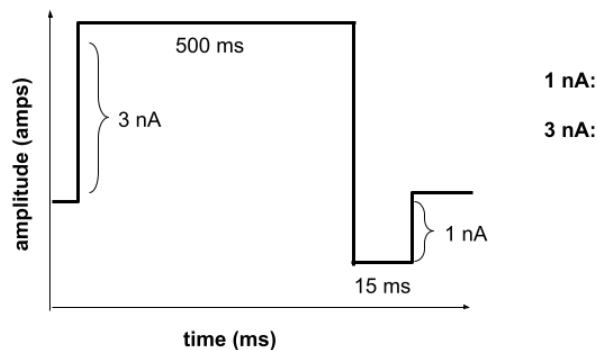
Electrode resistance = _____

If your resistance is higher than 100 M Ω , try the Ringer button to clear it. If this doesn't work after several tries, get a new pipette.

Configuring LabChart & stimulus pulses

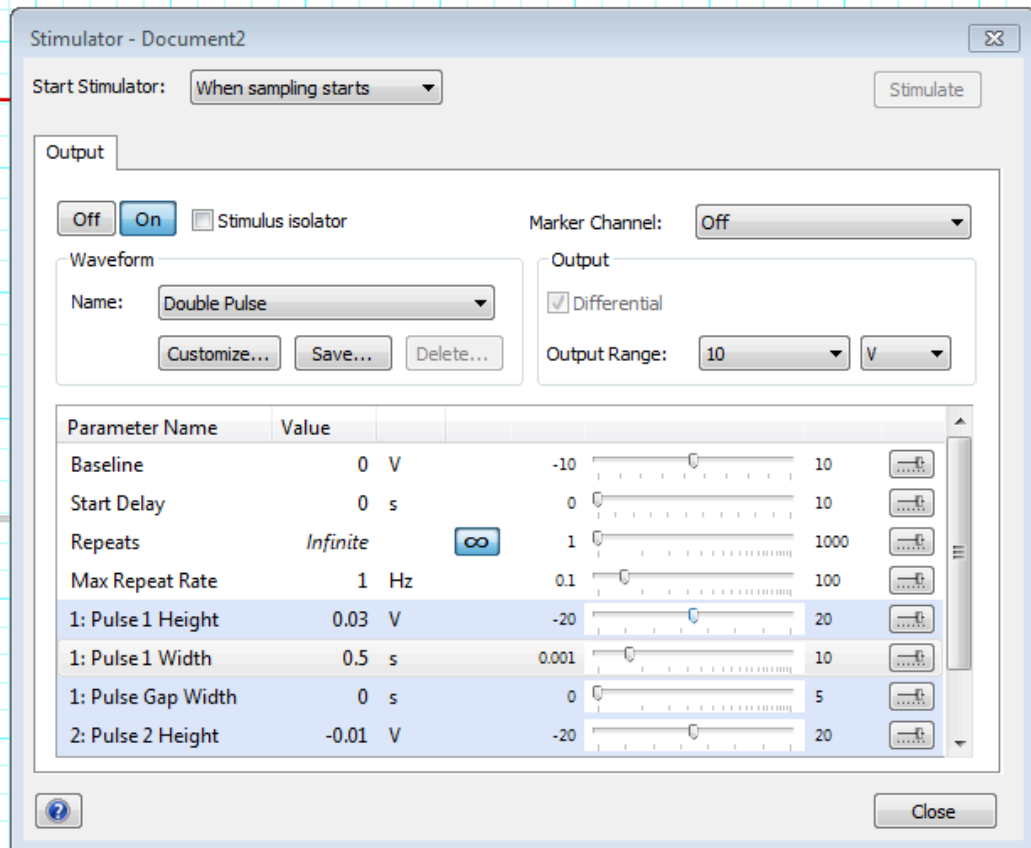
Note: While someone in your group is configuring the stimulus pulses, another group member should move onto Part II: Impaling a neuron.

Your configuration for LabChart will look similar to our leech recording session. Before we impale a cell, we'll also configure our pulses to look like this:



LabChart will tell the amplifier to send 1 nA for each 10 mV of stimulus given. Calculate the stimulator voltage we need for 1 and 3 nA and record above.

1. Open a new experiment in LabChart with the Leech Experiment settings.
2. Open the Stimulator window (Setup > Stimulator).
3. Change the wave type to "Double Pulse" setting in the Stimulator.
4. Set the parameters of your stimulus using the diagram below:



Hint: The waveform above is one positive pulse immediately followed by a negative pulse. You may need to change the possible range of these parameters before you can modify them to your needs.

Note: Change the Pulse Width of the second stimulus based on the diagram above. You can click on "Customize" to reassure yourself that you've configured the stimulus correctly.

5. Close the window when you're done.

Optional step: Directly monitor your stimulus waveform

To confirm that you've configured your stimulus correctly, you might want to actually monitor the stimulus waveform. But if you're running low on time, you should skip this step.

1. Unplug the EXTERNAL input to the amplifier and plug it into PowerLab Input 2 (using the DIN8 to BNC adapter). This will allow us to directly monitor the stimulus.
2. Change to Scope view, and change the Duration to 1 s.
3. Press Start and observe the shape of your waveform on your Stimulator channel (Channel 2). You may want to remove the Stimulus Marker from this channel if it's impeding your ability to see the waveform.

4. Plug the External cord back in. It should be as in the Setup diagram, with PowerLab Output (+) going into the EXTERNAL input on the amplifier.
5. Return to Chart view.

II. Impaling a neuron & filling your cell (1 hour)

Impaling a neuron

1. Impale a neuron of your choice, just as you did in the [Retzius Recording](#) experiment. You should try to target the cell you recorded from previously.
2. Briefly confirm its identity electrophysiologically. In other words, observe its firing rate and the shape of the action potential. If you're still not sure, inject current as we did in the recording experiment.

Filling your cell

Eject dye from the electrode by passing pulses of current into the cell. You should have the pulses configured already.

1. Make sure that the Current knob on your amplifier is on CONT (Continuous).
2. Turn the Stimulator ON and press Start.
3. Carefully turn off your surgery lights to avoid bleaching the dye.
4. Monitor the resistance of your electrode by observing the voltage change during each current pulse.
5. If the electrode starts to clog, your resistance (and change in voltage) will go up dramatically.
6. If your electrode looks like it's clogging, slowly remove it from the cell. Test the resistance (see above if you need a refresher on how to do this). Press the Ringer button on the amplifier, and test again. Repeat as necessary. If this does not solve your problem, make a new electrode (if time allows).
7. Every five minutes, turn off the Stimulator. Observe the resting potential of the cell to ensure that you're still in it. Turn on your lights and check to see if you can see any dye in the cell.
Note: Even if you can't see dye, that doesn't mean it's not there! We'll know for sure after fixing the ganglion and looking under the filters with the microscope.
8. Once you're content (or there is one hour left in class), slowly remove the electrode from the cell and stop recording in LabChart.
9. Save your data, just in case your dye fill doesn't work and you'd like to look at the data to figure out why.

III. Fixing & mounting your cells (1 hour)

Fixing & rinsing

In order to view the filled cells, we first need to fix them with paraformaldehyde.

1. Notify the IA or Instructor that you're ready. They'll perform the following steps under a fume hood:
2. Pour the leech saline from the dish into the buffer waste container.
3. Under the hood, add 2% PFA in 0.1 M phosphate buffer to your dish. Be sure that the ganglion is well submerged in the PFA. Fix for at least 20 minutes, while covered.
4. Dump the PFA from the plate into the PFA waste container.
5. Rinse your ganglia with phosphate buffer (PB) and dump into the buffer waste container.
6. Fill the dish with phosphate buffer and leave, covered, for 10 minutes.
7. When you get the ganglia back from the IA, dump out the PB, and repeat the PB wash twice (for a total of three 10 minute rinses).

Mounting on a slide ★

Once your leech ganglion is fixed and washed, it's ready to be prepared on a microscope slide for imaging.

1. Get a glass microscope slide and a coverslip.
2. Under the microscope, carefully remove the pins (4) from the leech ganglion dish with a pair of forceps. Leave the pins in the dish. The leech ganglion should be resting there.
3. Using the dropper, put a tiny drop of glycerol solution on the slide.
4. Gently pick up your ganglion and place it in the glycerol.
5. Add a coverslip. If you've never coverslipped before, ask for a demo.
6. Bring your slide to the fluorescent microscope to see what it looks like and take a photo!

Troubleshooting

Observation	Likely issue(s)	Possible solution
I can't see any cells, or they're really out of focus	Your lights are not properly aligned, or your focus needs to be adjusted	Try different light angles & configurations (1 light, or 2 lights), adjust the focus
My signal is really noisy	Something needs to be grounded	Make sure that everything metal within your Faraday cage is grounded, and that the Faraday cage itself is grounded
My resistance is really, really low (< 15 M Ω)	The tip of the electrode broke	Replace your electrode with a new one
My electrode resistance is too high (> 30 M Ω)	It is clogged	Remove the electrode from the cell (if applicable) and press the "Ringer" button to try to unclog it. If this doesn't help after several tries, get a new electrode.
There are no longer any spikes	Your electrode is no longer in the cell; your cell is very quiet; your cell died	Tap your electrode holder or gently move it closer to re-insert it into the cell
When I inject voltage, the reading says "Out of Range"	Your resistance is really high	(If applicable) remove the electrode from the cell, and use the Ringer button. Test your resistance again.

Drosophila Behavior & Optogenetics

Background & Goals

Drosophila as a model in neuroscience

With just over 100,000 neurons but incredible genetic tractability, the fruit fly is an important model organism in neuroscience. About 65% of human disease-associated genes have a homolog in *Drosophila melanogaster* (Ugur et al., 2016). Like many model organisms, fruit flies are easy to reproduce, cheap, and easy to take care of in a lab. On top of that, they've been studied for more than a century. *Drosophila* have been used to study learning, memory, vision, sleep, addiction, courtship, and aggression, among many other behaviors.

In these labs, we'll aim to characterize the gustatory, ethological, and locomotor behavior of transgenic fruit flies, solving a few mysteries along the way.

What's coming up

This module spans **two days** of lab:

- [Day 1: The Case of the Mislabeled Fly Vials](#) — three behavioral assays (proboscis extension reflex, ethological observation, and locomotor tracking) to characterize an unknown transgenic line.
- [Day 2: The Case of the Missing Methods](#) — design your own optogenetic experiment to figure out what the genetically modified neurons control.

In these labs you will:

- Operationalize and quantify the behavior of a fruit fly
- Use a computer vision program to track locomotor behaviors
- Identify different types of transgenic flies used for neuroscience research
- Design an optogenetic experiment to interrogate neural circuit function

Relevant references

Al-Anzi, B., Tracey, W. D., & Benzer, S. (2006). Response of *Drosophila* to wasabi is mediated by *painless*, the fly homolog of mammalian TRPA1/ANKTM1. *Current Biology* 16(10): 1034–1040.

Chen, Y.-C. D., & Dahanukar, A. (2020). Recent advances in the genetic basis of taste detection in *Drosophila*. *Cellular and Molecular Life Sciences* 77(6): 1087–1101.

Devineni, A. V., & Heberlein, U. (2009). Preferential ethanol consumption in *Drosophila* models features of addiction. *Current Biology* 19(24): 2126–2132.

Mandel, S. J., Shoaf, M. L., Bostwick, J. R., Combs, D. J., Trudeau, V. L., & Johnson, E. C. (2018). Behavioral aggression is regulated by *Drosophila* CCAP signaling. *Frontiers in Neural Circuits* 12: 45.

Nakamura, M., Baldwin, D., Hannaford, S., Palka, J., & Montell, C. (2002). Defective proboscis extension response (DPR), a member of the Ig superfamily required for the gustatory response to salt. *Journal of Neuroscience* 22(9): 3463–3472.

Trisal, S., VijayRaghavan, K., & Ramaswami, M. (2023). The proboscis extension reflex assay for evaluating taste responses in *Drosophila*. *Bio-protocol* 13(20).

Ugur, B., Chen, K., & Bellen, H. J. (2016). *Drosophila* tools and assays for the study of human diseases. *Disease Models & Mechanisms* 9(3): 235–244.

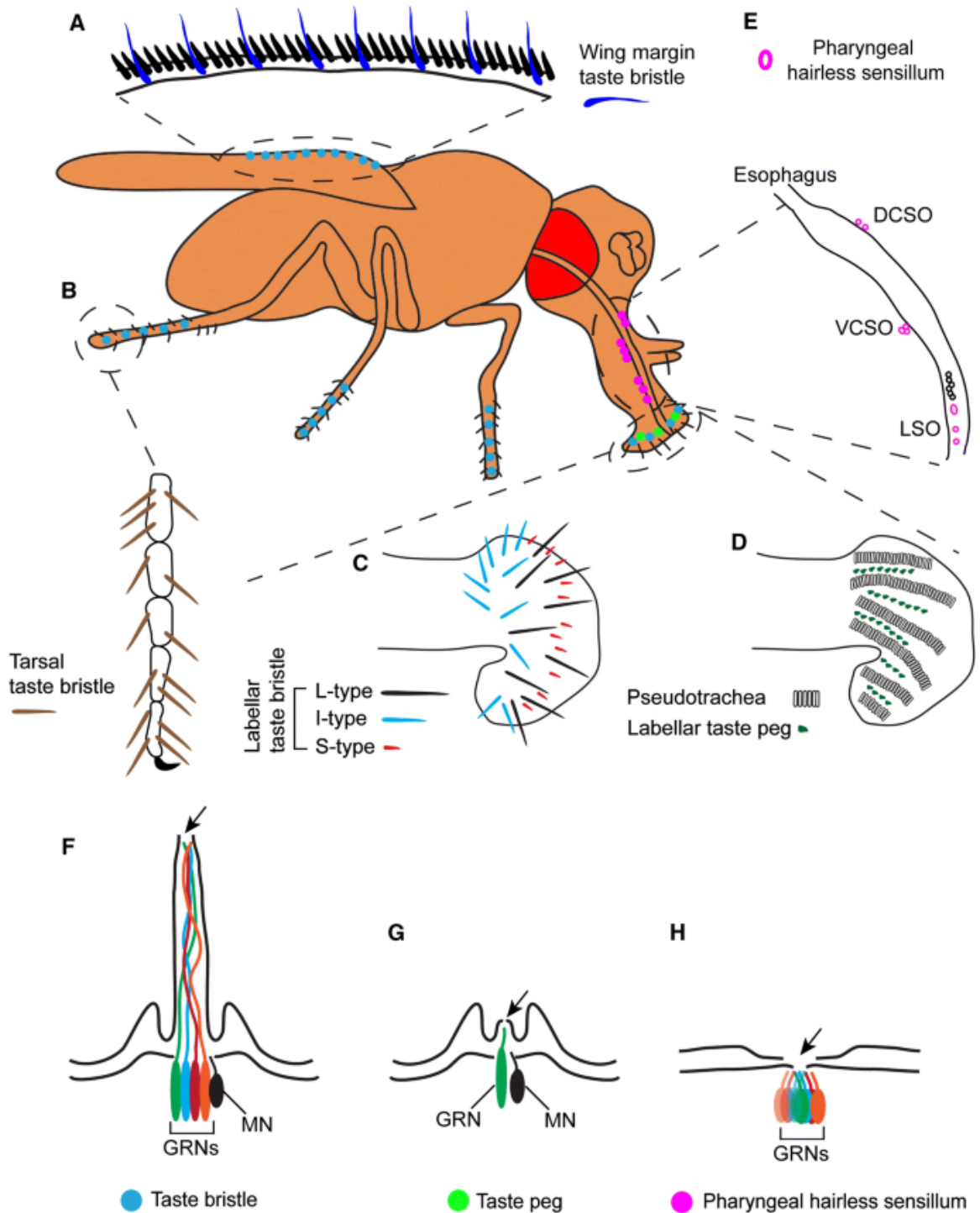
Day 1: The Case of the Mislabeled Fly Vials

Something terrible has happened in the lab. The flies you once knew, that were once perfectly organized and labeled, are now completely jumbled. You're pretty sure the flies are mutated, in one way or another. It's up to you to figure out what type of flies these are, based on three behavioral assays: a **proboscis extension reflex (PER) assay**, an **ethological survey**, and a **locomotion test**.

Everyone will receive one batch of wildtype flies, and one batch of an experimental group. Ultimately, **you will present your findings to the class** (see Canvas for details) so that we can determine the phenotype of our mutant flies.

Supplies	Solutions
Two vials of flies (~10 WT, ~10 Mutant/Knockout)	DI water
Dissecting microscope	100 mM sucrose solution
Paintbrush	40% EtOH in 100 mM sucrose solution
Large petri dish (lid or base)	
Clear cyanoacrylate glue	
4 mL transfer pipette	
Coverslips (6 per fly group)	
Paper towels & KimWipes	
Plastic "fly recovery" box	
Petri dish with sylgard bottom	
Magnetic stand & camera	

Test I. Proboscis Extension Reflex (PER) Assay



The **proboscis extension** is the reflexive extension of the mouthpart (proboscis) in response to taste stimuli. It is a fundamental feeding behavior that can be stimulated by exposing the antennae, labellar hairs, or tarsi (last segments of the leg) to an appetitive taste stimulus.

While a solution with a low concentration of ethanol is appetitive to fruit flies (serving as an olfactory cue that plant-material fermentation has begun), higher concentrations (>10% EtOH) can be aversive and toxic to fruit flies. In this assay, you'll compare your wildtype flies to a knockout group across three solutions: sucrose (an appetitive stimulus), DI water (a neutral control), and 40% EtOH in sucrose (an aversive stimulus).

Immobilization

1. Place fly tubes on ice for 2–3 minutes until flies are immobile.

Note: Keep a close eye on your flies and do not leave them on ice too long! As soon as they're not moving, you can transfer them.

2. Place the bottom of the petri dish upside down on ice. Wipe away condensation before placing flies on the dish.
3. Using a brush, *very gently* isolate one fly onto the petri dish and *carefully* orient the fly so it is dorsal-side up (Figure E below). Be gentle — the flies are very fragile.
4. Prepare a coverslip: using the transfer pipette, place a tiny drop (1 μ l, <2 mm in diameter) of glue on a coverslip.



5. Gently lower the coverslip onto the dorsal side of the fly, until just submerging the back and wings, immobilizing the fly in the glue (Figures F–G above).

Note: Only the back should be submerged in the glue. The head, proboscis, and at least one leg should be exposed — if any one of those parts is submerged, try again with another fly. If you've only submerged the wings, try flipping the coverslip over and using forceps to gently lower the back into the glue.

6. Place one damp paper towel in the plastic container on your lab bench. Ensure the box is dry. Place the coverslip in the box and close the container to regulate humidity.
7. Repeat with the remaining flies. You should prepare **5 wildtype** and **5 knockout** flies.
8. Allow the flies to recover for at least **1.5 hours** before testing. Complete the other behavioral tests during the recovery period.

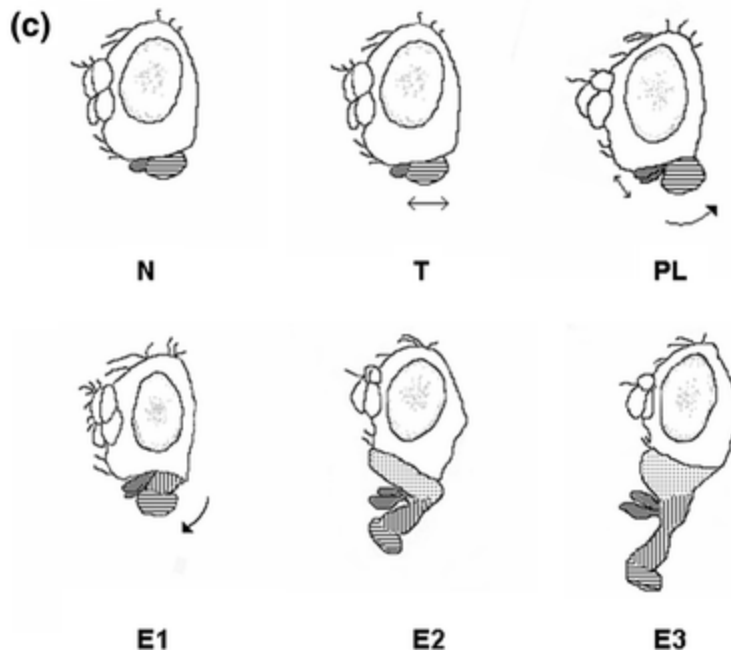
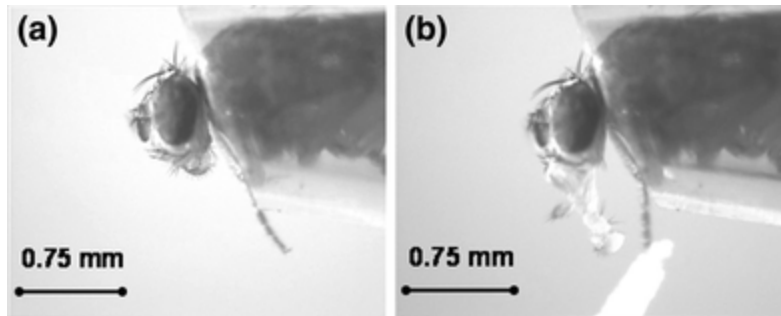
Note: Omit flies that do not show movement after 1.5 hours of recovery.

Running the PER assay

9. Choose one of your responsive, immobilized flies with accessible head and legs, and place it under the dissecting microscope. Set up your lighting so you can see the legs

and proboscis.

10. Dip the corner of a KimWipe in the sucrose solution and gently touch the tarsi with the KimWipe for 3 seconds.



a. Score the response in **Data Table 1**. Assign a **1** for proboscis extension (right image above) and a **0** for no extension (left image above).

b. Wait 3 seconds or until the proboscis retracts.

c. Repeat for **10 trials**.

11. Repeat using **DI water**.

12. Repeat using **40% EtOH in sucrose**.

Data Table 1: PER Responses

Genotype	Trial	Sucrose	DI Water	40% EtOH in Sucrose
Wildtype	1			
Wildtype	2			
Wildtype	3			
Wildtype	4			
Wildtype	5			
Wildtype	6			
Wildtype	7			
Wildtype	8			
Wildtype	9			
Wildtype	10			
Knockout	1			
Knockout	2			
Knockout	3			
Knockout	4			
Knockout	5			
Knockout	6			
Knockout	7			
Knockout	8			
Knockout	9			
Knockout	10			

Reflection Question: Identify the positive and negative controls in this experiment. With these controls in mind, are your findings as expected?

Test II. Ethological approach to behavior

It's possible your flies do something different that's *not* captured by a standard behavioral assay. The best way to figure this out is to watch the flies, choose several behaviors to quantify, and see if those behaviors differ between your experimental and wildtype flies.

Choosing behaviors to quantify

Your next goal for today will be to quantify **three** different behaviors that your flies perform. It is up to you to define and operationalize these behaviors.

1. Freeze both groups of flies for 2–3 minutes until they stop moving.
2. Isolate three flies of each type in separate petri dishes.
Note: Make sure the flies are completely awake and moving before moving on to the next step. This could take about 10 minutes.
3. Take a few minutes to observe both batches of flies.
4. Choose three different behaviors that the flies do, and record them below. You also need to provide a definition for each behavior.

Table 1.

Behavior name	Definition	# Instances in WT	# Instances in Mutant

5. For 5 minutes, each person in your group should observe one behavior and record the number of instances those behaviors happen in the WT flies. Record the number of instances in the table above.
6. Do the same for the experimental group.

Test III. Recording locomotor behavior

For the final test, you'll record the movement of your two different types of flies in a petri dish and use an automated tracking program to track the velocity and heading of your flies. Your mutants might have altered locomotor behavior — this will help you test that.

Set up your camera

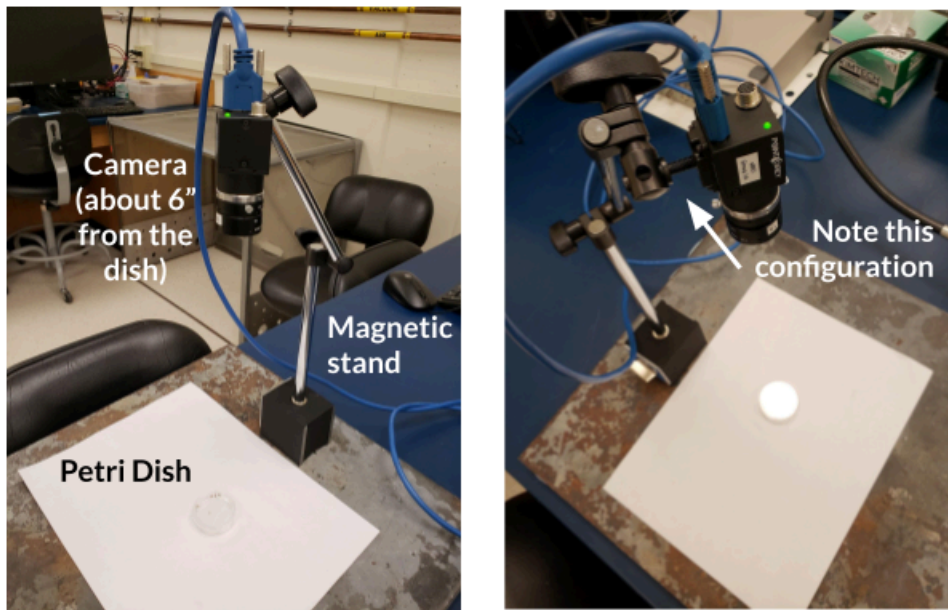


Figure 1. Fly locomotor recording setup.

First, we need to arrange the camera and petri dish so that you can get everything in frame, in focus, and well lit.

1. Ensure that your camera is connected via the USB cord to the computer. The camera should be plugged into a USB 3.0 slot (the blue ones).
2. Set up your camera and petri dish as in the figure above. You want your camera to be about 6 inches from the petri dish.
3. On the computer, open **Point Grey FlyCap2**.
4. Make sure that it recognizes a "Flea3" camera, and press OK.
5. Adjust the **focal length** knob on your camera (closest to end, ranges from infinity to 0.1) in order to get the image in focus.
6. Adjust the image on your screen using the toolbar at the top:

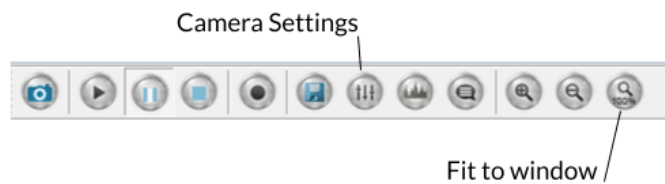
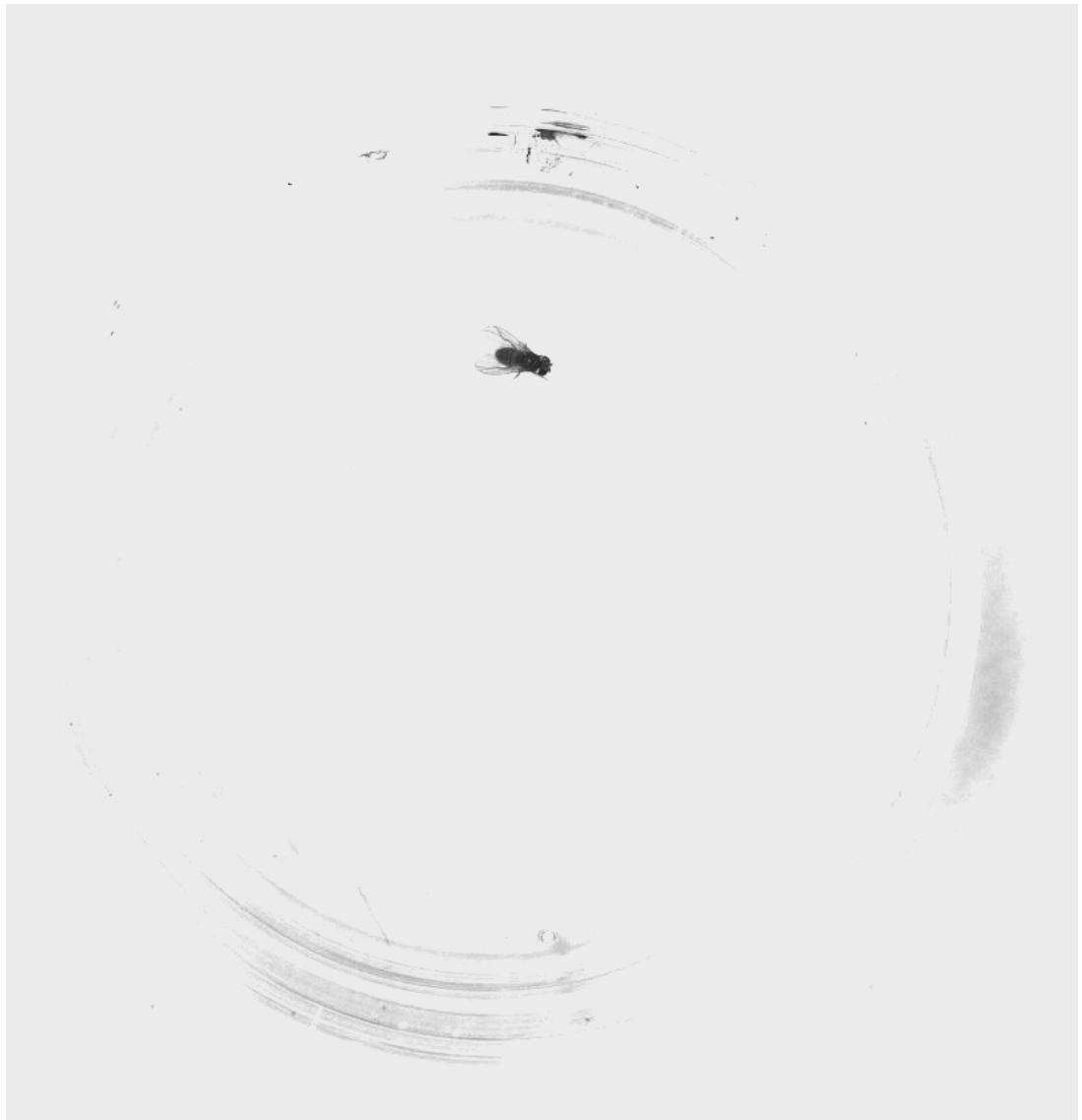


Figure 2. FlyCap menu bar.

7. Isolate one fly in your petri dish.
8. Click on the button to open up the Camera settings (e.g., exposure).
 - a. Uncheck *auto* for brightness, exposure, and shutter.
 - b. Increase the exposure almost to the point where your image is overexposed. You want the flies to stand out very clearly against the white background, and the edges of the petri dish to be invisible:



9. Ensure that you can easily see your fly throughout all areas of the arena.

Note: Optimizing the way your fly recording looks will make the tracking *much* easier. Make sure there are no glares from lights in the room, no dark spots on the petri dish, and clean off any fingerprints on the glass. **Every light and dark spot matters, and anything can throw off the tracking.**

Set up your flies & record

Recording Settings

Save Filename
 Filename: C:\Users\ajuavine\Videos\fly_test
 A timestamp, sequence number and file extension based on output type will be appended Browse

Saving Options
☐ Capture 0 frames*
☐ Capture 1 out of every frames from a run of frames*
☒ Capture for 60000 ms
☐ Capture once out of every ms from a run of ms*
 * Specify zero to capture until manually stopped

Recording Mode
☒ **Buffered:** Frames are buffered in physical memory while recording. Recording will stop if physical memory usage hits 95%
☐ **Streaming:** Frames are not buffered in system memory while recording. Dropped frames may occur depending on the speed of the PC and the current camera/recording settings.

☐ Only save corrupt frames when capturing

Images Videos

Video Recording Type: M-JPEG
 Frame Rate: Use Camera Frame Rate
 AVI Split Size: 0 MB (0 indicates no file split)
 M-JPEG Options
 JPEG Compression Quality (1-100): 75

Good Frames:	1198	Available Physical Memory:	12522.50 MB
Corrupt Frames:	0	Total Physical Memory:	16304.59 MB
Buffered Frames:	0	Usage:	23%
Saved Frames:	1198		

Hide Recording is stopped Start Recording

1. When you're ready to record, go to **File > Capture Image or Video**.
2. A dialogue will pop up. Set up a 60 second recording with the following settings:
 - **Filename:** browse to create your own (make sure you label it as WT or Knockout). **SAVE YOUR FILE LOCALLY**, not on your username server. **Your video file will not save correctly if you do not do this.**
 - ☒ Capture for 60000 ms
 - ☒ Buffered
 - (Click on **Videos** tab)
 - **Video Recording Type:** M-JPEG

- **Frame Rate:** 15.0
 - **AVI Split Size:** 0
 - **JPEG Compression Quality:** 75
3. Press **Start Recording**, and wait while the program records the video.
Note: This window will not close, but the message at the bottom will change.
 4. When it is done, **check that the video recorded correctly by finding the file and opening it.**
Note: If it didn't record correctly, close FlyCap, unplug the camera, and retry.
 5. For the first fly you record, **move on to the next step to ensure that your video works well for tracking. If the tracking doesn't work well, you'll need to adjust your lighting and/or image acquisition.**
 6. When you're confident the tracking works well with your videos, obtain videos for three flies from each group (you've already done one).
 7. When you're done, save your videos in a clouded location for safekeeping.
 8. Close FlyCap.

Quantify your locomotor behavior

Now that you have videos of your specimen, you need to quantify the locomotor behavior. You can do so using this notebook, which will generate means and standard deviations for each video:

<https://bipn145.github.io/FlyTracking/ColabNotebook.html>

Record those values in the table below (you'll ultimately want to plot these).

Table 2. Comparison of mean velocities for control and mutant flies.

	Control	Mutant
Mean		
SEM		

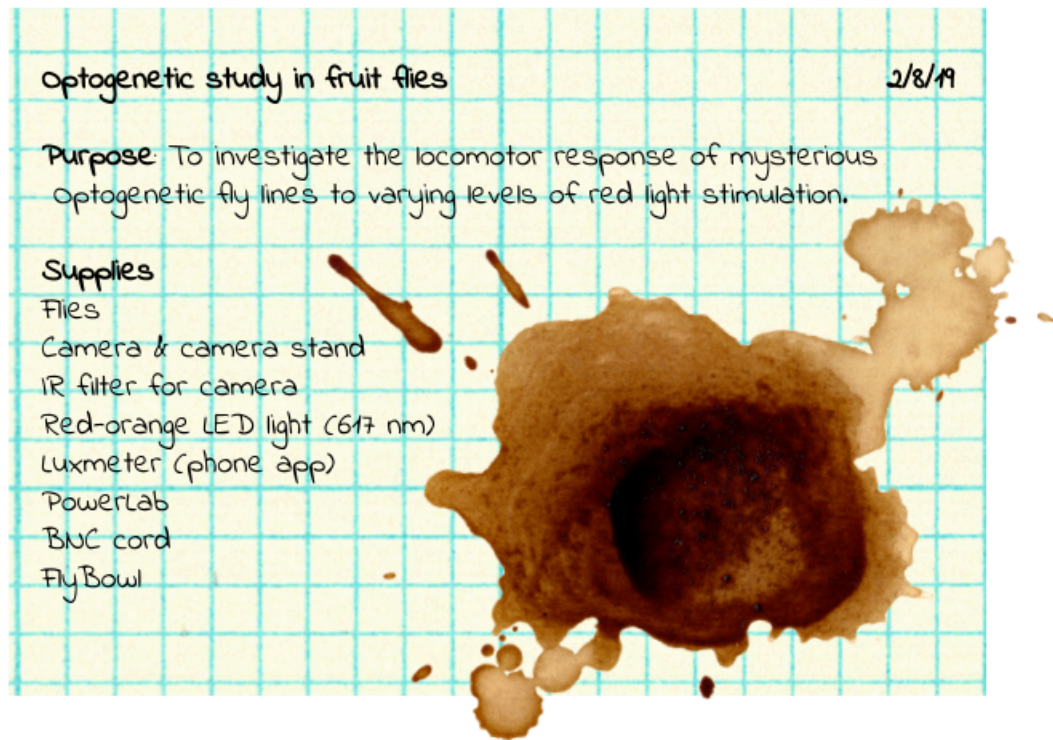
Troubleshooting

Observation	Likely issue(s)	Possible solution
The camera output is black	Lens cap is on	Remove the lens cap
Camera not recognized	Camera not plugged in	Plug in the camera
Video file is not recognized in MATLAB or the movie is the wrong length/size	You wrote the video file to the server	You need to re-record your video but with the file saved directly to the computer locally
Fly does not extend proboscis to sucrose	Fly is dehydrated or improperly mounted	Check humidity in recovery box; verify proboscis and at least one leg are not stuck in glue
All flies look the same in tracking video	Lighting/exposure issues	Increase exposure so flies stand out as dark spots on a fully white background

Day 2: The Case of the Missing Methods

Just when you thought everything was back to normal in the lab, a remarkably inconveniently placed coffee spill completely drenches several pages of the notes given to you from your mentor.

Unfortunately, those pages detailed all of the methods that you needed in order to perform your experiment. You've managed to save the supplies list, at least:



Hope is not lost. You've got some supplies, and you suspect that this study had something to do with the role of the genetically-modified neurons. Certain neurons have optogenetic channels, but what do those neurons actually control?

It might take a little bit of research to refresh your memories about those transgenic flies, but that's easy enough. And you've done at least some fly behavior before — you're pretty much the fly whisperer.

You can re-write the protocol and get some interesting results, can't you?

Tips for designing your experiment

You can control the intensity of the light source through LabChart by following these steps:

1. Plug the BNC input on the light into the **OUTPUT** of the PowerLab using a BNC cable.
2. Program the stimulator (**Setup > Stimulator**) to send different pulses with heights from 2V – 10V. You should notice that the intensity of the light changes with more voltage.

Do not stimulate your flies at higher than 1 Hz frequency or with stimulus pulses longer than 0.5 seconds.

3. Measure the intensity of the light using a phone app luxmeter (we recommend *Lux Light Meter Free*). You should measure the light intensity **from the same distance** as you are using it in the experiment.

Don't forget about control groups! It may help to compare these flies to each other, as well as to wildtype flies. We have wildtype flies if you need them.

You will need to put the red light **within several inches** of your flies in order to see an effect of the stimulation.

Fly Tracker (Local)

In this lab, you'll use a Python script to track the position and velocity of a fruit fly from video recordings. The script will detect the fly against a light background, track its movement frame-by-frame, and output position and velocity data as CSV files, along with plots.

At the end of this notebook, you'll be able to:

- Run the fly tracker script on your own video recordings
- Load and visualize tracking and velocity data in Python
- Customize your plots for your lab report

Part 1: Running the Fly Tracker Script

The fly tracker is a Python script that you'll run from the command line. Follow the steps below to get set up.

Step 1: Open Anaconda Prompt

1. Click the **Windows search bar** (bottom-left of your screen) and type **Anaconda**.
2. Click **Anaconda Prompt** to open it. You should see a black terminal window.

Note

This is intended to be run on the BIPN 145 Windows lab computers in York 1310. If you are running this on your own Mac computer, you can search for Python and run it that way. If you're running it on your own Windows computer, you'll need to download Python or Anaconda.

Step 2: Download the Scripts

In the Anaconda Prompt, first use the following to move into *your* folder. Replace USER with the UCSD username (not including @ucsd.edu) for the person that is logged into the computer.

Tip

Copying and pasting into the Anaconda Prompt: You can copy commands from this page by clicking the copy button () at the top-right of each code block and using Ctrl+V to paste into the command line.

```
cd C:\Users\USER
```

Then, run the following commands to download the setup and fly tracker scripts:

```
curl -o setup.py https://raw.githubusercontent.com/ajuavinett/fly_tracker/master
```

```
curl -o BIPN145_flytrack.py https://raw.githubusercontent.com/ajuavinett/fly_tra
```

If you already have older versions of these scripts, this will overwrite them with the latest versions. You can also download the scripts manually from the [fly_tracker GitHub page](#).

Step 3: Install Required Packages

Run the setup script to install the required Python packages. You only need to do this **once** per computer:

```
python setup.py
```

You should see a message confirming that each package is installed. Once you see "Setup complete!", you're ready to go.

Step 4: Know Where Your Video File(s) Are

Make sure you know where your fly video file(s) (`.avi`, `.mp4`, etc.) are saved on your computer. When you run the script, a file picker window will pop up and you can navigate to them from there.

Step 5: Run the Script

In the Anaconda Prompt, type:

```
python BIPN145_flytrack.py
```

The script will walk you through everything interactively:

1. **Select your video(s)** — A file picker dialog will pop up. Navigate to your video file(s), select them, and click Open.
2. **Draw the ROI** — A window will pop up showing the first frame of your video. **Click and drag** to draw a rectangle around the dish/arena (and *only* around the dish – make it as snug as possible!), then press **Enter** or **Space** to confirm. Press **C** to cancel and redraw.
3. **Wait for tracking** — The script will process each frame. You'll see progress updates (10%, 20%, etc.). This will take a few minutes to run per video.
4. **View results** — When done, plots will appear will save automatically along with CSV files containing the tracking results. You can either use the plots that are saved or modify them (see below).

Step 6: Find Your Output Files

After the script finishes, you'll find two CSV files for each video in the same folder as your video:

File	Contents
<code>yourvideo_tracking.csv</code>	Time (s), X position (cm), Y position (cm)
<code>yourvideo_velocity.csv</code>	Time (s), Velocity (mm/s)

If you need to edit them, you can open these in Excel, Google Sheets, or load them into Python (see below).

Part 2: Working with Your Data in Python (Optional)

The cells below will help you load your CSV output files and create customized plots in case you need to adjust anything after running the tracking. This is useful if you want to adjust axis labels, colors, or combine data from multiple files for your lab report.

Task: Run the cell below to import the packages we'll need.

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.collections import LineCollection

print('Packages imported!')
```

Upload your CSV files

If you're running this notebook in **Google Colab**, run the cell below to upload your tracking and velocity CSV files. If you're running locally, you can skip this cell and just set the file paths directly.

Task: Run the cell below and upload your `_tracking.csv` and `_velocity.csv` files.

```

try:
    from google.colab import files
    uploaded = files.upload()
    filenames = list(uploaded.keys())
    tracking_file = [f for f in filenames if 'tracking' in f][0]
    velocity_file = [f for f in filenames if 'velocity' in f][0]
    print(f'Tracking file: {tracking_file}')
    print(f'VeLOCITY file: {velocity_file}')
except ImportError:
    # Running locally – set your file paths here
    tracking_file = 'yourvideo_tracking.csv' # <-- change this
    velocity_file = 'yourvideo_velocity.csv' # <-- change this
    print(f'Using local files: {tracking_file}, {velocity_file}')

```

Load the data

Now let's load both CSV files using `numpy`. Each file has a header row, so we skip it with `skiprows=1`.

Task: Run the cell below to load your data.

```

# Load tracking data: columns are Time (s), X (cm), Y (cm)
tracking = np.loadtxt(tracking_file, delimiter=',', skiprows=1)
time_pos = tracking[:, 0]
x_pos = tracking[:, 1]
y_pos = tracking[:, 2]

# Load velocity data: columns are Time (s), Velocity (mm/s)
vel_data = np.loadtxt(velocity_file, delimiter=',', skiprows=1)
time_vel = vel_data[:, 0]
velocity = vel_data[:, 1]

print(f'Tracking data: {len(time_pos)} frames')
print(f'VeLOCITY data: {len(time_vel)} time bins')
print(f'Mean velocity: {np.nanmean(velocity):.2f} mm/s')
print(f'SD velocity: {np.nanstd(velocity):.2f} mm/s')

```

Plot the fly path

The plot below shows the fly's path through the arena, color-coded by time. Earlier positions are shown in purple/blue, and later positions in yellow/green.

Task: Run the cell below to plot the fly path. Edit the axis labels, title, or colormap (`cmap`) if you'd like to customize it.

```

# Set the diameter of your dish (should match what you used in the script)
diameter = 4 # cm

fig, ax = plt.subplots(figsize=(6, 6))

# Create color-coded path
valid = ~np.isnan(x_pos) & ~np.isnan(y_pos)
points = np.array([x_pos[valid], y_pos[valid]]).T.reshape(-1, 1, 2)
segments = np.concatenate([points[:-1], points[1:]], axis=1)

lc = LineCollection(segments, cmap='viridis', linewidth=2)
lc.set_array(time_pos[valid][:-1])
ax.add_collection(lc)

ax.set_xlim(0, diameter)
ax.set_ylim(diameter, 0) # Inverted to match video orientation
ax.set_aspect('equal')

# --- Customize these labels for your report ---
ax.set_xlabel('X-coordinate (cm)', fontsize=12)
ax.set_ylabel('Y-coordinate (cm)', fontsize=12)
ax.set_title('Fly Path', fontsize=14)

cbar = fig.colorbar(lc, ax=ax, orientation='horizontal', pad=0.1)
cbar.set_label('Time (s)')

plt.tight_layout()
plt.show()

```

Plot velocity over time

The plot below shows how the fly's velocity changes over the course of the recording.

Task: Run the cell below to plot the velocity trace. Edit the labels, colors, or axis limits as needed.

```

fig, ax = plt.subplots(figsize=(10, 4))

ax.plot(time_vel, velocity, linewidth=1.5, color='steelblue')

# --- Customize these for your report ---
ax.set_xlabel('Time (s)', fontsize=12)
ax.set_ylabel('Velocity (mm/s)', fontsize=12)
ax.set_title('Fly Velocity Over Time', fontsize=14)

# Uncomment the line below to set custom axis limits:
# ax.set_xlim(0, 60) # e.g., first 60 seconds
# ax.set_ylim(0, 20) # e.g., max 20 mm/s

plt.tight_layout()
plt.show()

print(f'Mean velocity: {np.nanmean(velocity):.2f} mm/s')
print(f'SD velocity: {np.nanstd(velocity):.2f} mm/s')

```

Compare multiple flies *(optional)*

If you have velocity data from multiple flies, you can plot them together and compute summary statistics.

Task: Upload additional velocity CSV files and add their filenames to the list below.

```

# Add your velocity CSV filenames here
velocity_files = [
    'fly1_velocity.csv', # <-- change these
    'fly2_velocity.csv',
    # 'fly3_velocity.csv',
]

fig, ax = plt.subplots(figsize=(10, 5))
all_means = []

for i, vf in enumerate(velocity_files):
    data = np.loadtxt(vf, delimiter=',', skiprows=1)
    t = data[:, 0]
    v = data[:, 1]
    ax.plot(t, v, linewidth=1.5, label=f'Fly {i + 1}')
    all_means.append(np.nanmean(v))

ax.set_xlabel('Time (s)', fontsize=12)
ax.set_ylabel('Velocity (mm/s)', fontsize=12)
ax.set_title('Velocity Comparison', fontsize=14)
ax.legend()
plt.tight_layout()
plt.show()

print(f'\nMean velocity across flies: {np.mean(all_means):.2f} mm/s')
print(f'SD across flies: {np.std(all_means):.2f} mm/s')

```


About this Notebook

This notebook was created by [Ashley Juavinett](#) for classes at UC San Diego. The fly tracker script is based on MATLAB code by Jeff Stafford.

Fly Tracker (Colab)

This notebook tracks a fruit fly against a light background in a video, calculates its position and velocity over time, and produces path and velocity plots.

Notebook steps

1. Run the **Setup** cell to import packages.
2. **Mount your Google Drive** and copy the path to your video file.
3. Create the **Helper Functions**.
4. Run the ROI cell and **draw your ROI** on the first frame, then confirm it.
5. Run the **Track Fly** cell and wait for it to finish.
6. Run the **Plot** and **Velocity** cells to view results.
7. Re-run steps 3-7 if you would like to track an additional video.

Setup

```
##@title Install & import packages { display-mode: "form" }
import numpy as np
import cv2
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
from matplotlib.collections import LineCollection
from scipy.spatial.distance import euclidean
from google.colab import files, output
from IPython.display import display, clear_output, HTML, Image as IPIImage
import ipywidgets as widgets
import os
import base64

# Initialize result containers – each is created independently so adding
# new lists in future won't require a full session restart
# You can uncomment the following to reset all results if needed:
#all_corrected = []
#all_velocity = []
#all_fps = []
#all_video_paths = []
#print('Results cleared – ready to process new videos.')
```

```
if 'all_corrected' not in dir(): all_corrected = []
if 'all_velocity' not in dir(): all_velocity = []
if 'all_fps' not in dir(): all_fps = []
if 'all_video_paths' not in dir(): all_video_paths = []
print(f'Ready. {len(all_corrected)} video(s) processed so far.')
```

```
print('All packages loaded successfully!')
```

Mount your drive

Run the cell below to mount your drive, which should contain your video files.

If you'd prefer to upload your videos, you can do that instead (and do not need to run the cell below. See instructions [here](#).)

```
from google.colab import drive
drive.mount('/content/drive')
```

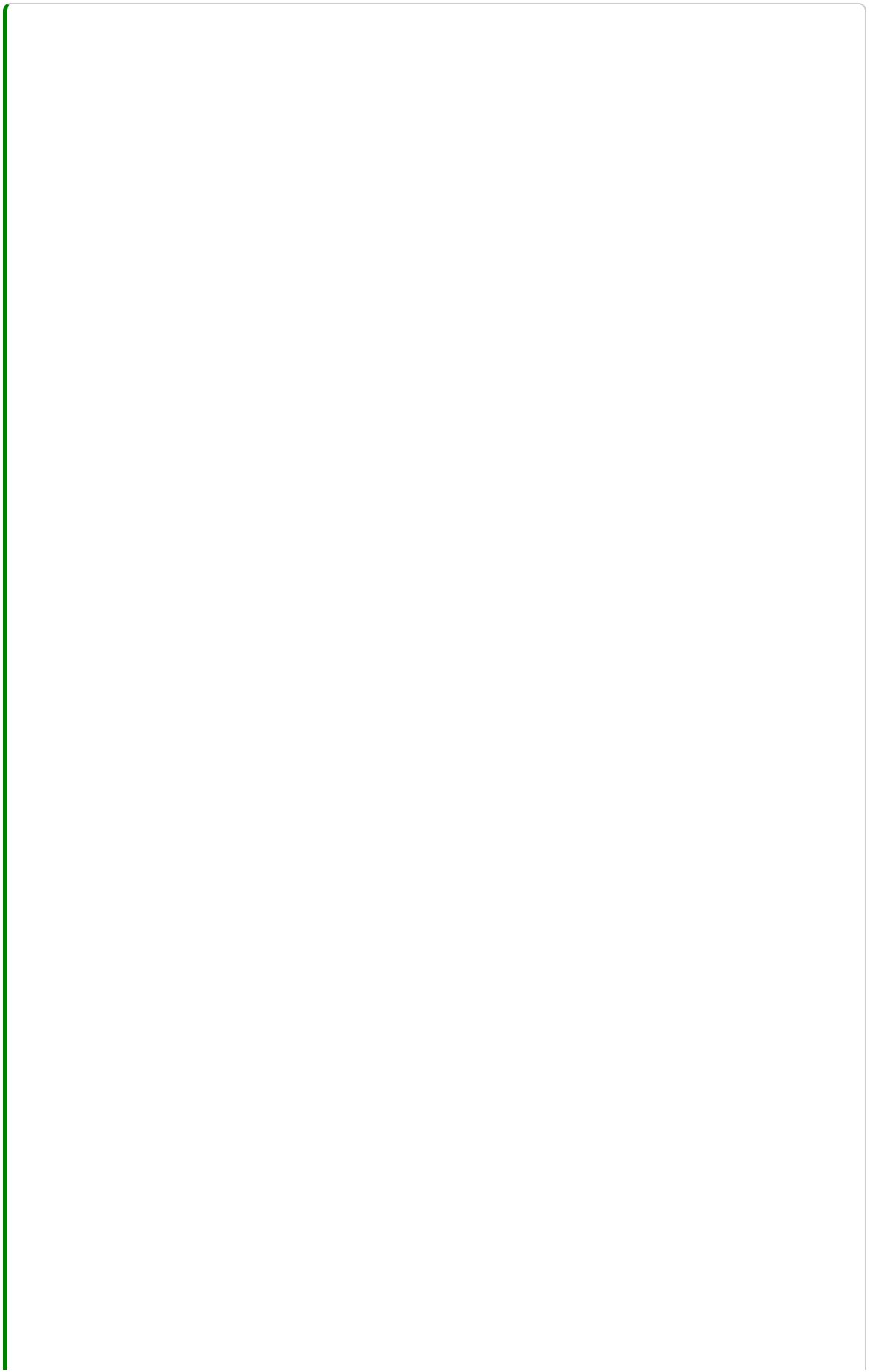
Select Video File

Update the path below to your video file. You'll process one video at a time, and results will accumulate in `all_corrected` and `all_velocity`.

Tip: In the window at left, you can use the three vertical dots to copy and paste the path to your video file.

```
# Set the current video file to process
current_video = '/content/drive/MyDrive/video1.avi'

if os.path.exists(current_video):
    size_mb = os.path.getsize(current_video) / 1e6
    print(f'File found: {os.path.basename(current_video)} ({size_mb:.1f} MB)')
else:
    print(f'File not found: {current_video}')
    print()
    print('Troubleshooting tips:')
    print('  1. Make sure you have run the "Mount your drive" cell above.')
    print('  2. In the left sidebar, click the folder icon to open the file browser.')
    print('  3. Navigate to your video file, right-click it, and select "Copy path to clipboard".')
    print('  4. Paste the copied path above to replace the current_video value.')
    print()
    # Show video files in the same folder if it exists, to help spot typos
    folder = os.path.dirname(current_video)
    if os.path.exists(folder):
        videos = [f for f in os.listdir(folder)
                  if f.lower().endswith(('.avi', '.mp4', '.mov', '.mkv', '.mj2'))]
        if videos:
            print(f'Video files found in {folder}:')
            for v in sorted(videos):
                print(f'  {os.path.join(folder, v)}')
        else:
            print(f'No video files found in {folder} – check that you are looking in the right folder.')
    else:
        print(f'Folder not found: {folder}')
        print('  Make sure your Drive is mounted and the path is correct.')
```



```

#@title Define helper functions (fly_finder, dist_filter, interpolate_pos) { di

# --- Parameters ---
frame_rate = None # auto-detect from video (set e.g. frame_rate = 30 to overri
diameter = 4 # dish diameter in centimeters
search_size = 20 # fly search window size in pixels
per_pixel_threshold = 1.5
bin_size = 1 # velocity bin size in seconds
# -----

def fly_finder(roi_image, half_search, threshold, flip=True):
    """
    Find a fly (dark region) in a grayscale image.
    Locates the darkest pixel, retrieves a search area around it,
    and finds the center of pixel intensity.

    Returns (x, y) position or (NaN, NaN) if not found.
    """
    if flip:
        val = np.nanmin(roi_image)
    else:
        val = np.nanmax(roi_image)

    ys, xs = np.where(roi_image == val)
    xpos = np.mean(xs)
    ypos = np.mean(ys)

    h, w = roi_image.shape
    left = max(int(round(xpos) - half_search), 0)
    right = min(int(round(xpos) + half_search), w - 1)
    top = max(int(round(ypos) - half_search), 0)
    bottom = min(int(round(ypos) + half_search), h - 1)

    search_area = roi_image[top:bottom+1, left:right+1].astype(np.float64)

    if flip:
        search_area = 255.0 - search_area

    total = np.sum(search_area)

    if total >= threshold:
        x_indices = np.arange(search_area.shape[1])
        y_indices = np.arange(search_area.shape[0])
        x = np.sum(search_area @ x_indices) / total + left
        y = np.sum(search_area.T @ y_indices) / total + top
        return x, y
    else:
        return np.nan, np.nan

def dist_filter(array, tele_dist_threshold, num_avg=5):
    """
    Teleport filter: removes spurious points where fly position
    jumps far from the mean of surrounding frames.

    array: Nx3 array [time, x, y]
    """
    filtered = array.copy()
    n = len(filtered)
    x = filtered[:, 1]

```

```

y = filtered[:, 2]

def rolling_nanmean(arr, window, forward=False):
    """Rolling mean over 'window' preceding (or following) valid elements."""
    valid = (~np.isnan(arr)).astype(np.float64)
    filled = np.where(np.isnan(arr), 0.0, arr)
    cs_val = np.concatenate([[0.0], np.cumsum(filled)])
    cs_cnt = np.concatenate([[0.0], np.cumsum(valid)])
    idx = np.arange(n)
    if forward:
        starts = idx + 1
        ends = np.minimum(n, idx + 1 + window)
    else:
        starts = np.maximum(0, idx - window)
        ends = idx
    sum_v = cs_val[ends] - cs_val[starts]
    cnt_v = cs_cnt[ends] - cs_cnt[starts]
    with np.errstate(invalid='ignore', divide='ignore'):
        return np.where(cnt_v > 0, sum_v / cnt_v, np.nan)

last_x = rolling_nanmean(x, num_avg)
last_y = rolling_nanmean(y, num_avg)
next_x = rolling_nanmean(x, num_avg, forward=True)
next_y = rolling_nanmean(y, num_avg, forward=True)

dist_last = np.sqrt((x - last_x) ** 2 + (y - last_y) ** 2)
dist_next = np.sqrt((x - next_x) ** 2 + (y - next_y) ** 2)

valid_pts = ~np.isnan(x)
tele_mask = valid_pts & (
    (dist_last > tele_dist_threshold) | (dist_next > tele_dist_threshold)
)
tele_mask[:num_avg] = False
tele_mask[n - num_avg:] = False

filtered[tele_mask, 1:3] = np.nan
tele_count = int(tele_mask.sum())

edge_indices = list(range(0, min(5, n - 1))) + list(range(max(0, n - 6), n))
for idx in edge_indices:
    if np.any(np.isnan(filtered[idx, 1:3])) or np.any(np.isnan(filtered[idx + 1, 1:3])):
        continue
    if euclidean(filtered[idx, 1:3], filtered[idx + 1, 1:3]) > tele_dist_th:
        filtered[idx, 1:3] = np.nan
        tele_count += 1

print(f'{tele_count} points removed by the teleportation filter.')
return filtered

def interpolate_pos(array, inter_dist_threshold):
    """
    Linearly interpolate fly position between NaN gaps,
    as long as the gap endpoints are within inter_dist_threshold.

    array: Nx3 array [time, x, y]
    """
    result = array.copy()
    interp_count = 0

    x = result[:, 1]
    y = result[:, 2]

```

```

nan_mask = np.isnan(x)

if not nan_mask.any():
    print('0 points recovered through interpolation.')
    return result

padded = np.concatenate([[False], nan_mask, [False]])
diff = np.diff(padded.astype(np.int8))
starts = np.where(diff == 1)[0]
ends = np.where(diff == -1)[0] - 1

for s, e in zip(starts, ends):
    last_idx = s - 1
    next_idx = e + 1
    if last_idx < 0 or next_idx >= len(result):
        continue
    last_pt = result[last_idx, 1:3]
    next_pt = result[next_idx, 1:3]
    if np.any(np.isnan(last_pt)) or np.any(np.isnan(next_pt)):
        continue
    if euclidean(last_pt, next_pt) <= inter_dist_threshold:
        gap = next_idx - last_idx
        fracs = np.arange(1, gap) / gap
        result[s:e + 1, 1] = last_pt[0] + fracs * (next_pt[0] - last_pt[0])
        result[s:e + 1, 2] = last_pt[1] + fracs * (next_pt[1] - last_pt[1])
        interp_count += e - s + 1

print(f'{interp_count} points recovered through interpolation.')
return result

def boundary_filter(array, diameter):
    """
    Remove points that fall outside the circular dish.
    """
    filtered = array.copy()
    cx, cy = diameter / 2, diameter / 2
    radius = diameter / 2
    x = filtered[:, 1]
    y = filtered[:, 2]
    dist_from_center = np.sqrt((x - cx) ** 2 + (y - cy) ** 2)
    outside = dist_from_center > radius
    outside = outside & ~np.isnan(x)
    filtered[outside, 1:3] = np.nan
    count = np.sum(outside)
    if count > 0:
        print(f' {count} points removed by the boundary filter (outside dish).')
    return filtered

def displacement_filter(array, multiplier=5):
    """
    Remove points where the frame-to-frame displacement is much larger
    than the median displacement. This catches sudden jumps that the
    teleport filter might miss (e.g., runs of bad frames).
    """
    filtered = array.copy()
    x = filtered[:, 1]
    y = filtered[:, 2]

    dx = np.diff(x)
    dy = np.diff(y)

```

```

displacements = np.sqrt(dx ** 2 + dy ** 2)

valid_disp = displacements[~np.isnan(displacements)]
if len(valid_disp) == 0:
    return filtered
median_disp = np.nanmedian(valid_disp)

if median_disp == 0:
    return filtered

threshold = median_disp * multiplier
count = 0

for i in range(len(displacements)):
    if np.isnan(displacements[i]):
        continue
    if displacements[i] > threshold:
        filtered[i + 1, 1:3] = np.nan
        count += 1

if count > 0:
    print(f' {count} points removed by the displacement filter (>{multiplier}x median displacement)')
return filtered

def process_video(video_path, roi, diameter, frame_rate, search_size, per_pixel):
    """
    Process a single fly video and return the corrected position array.
    Returns (Nx3 array [time_s, x_cm, y_cm], frame_rate).
    """
    height = diameter
    width = diameter
    roi_x, roi_y, roi_w, roi_h = roi

    cap = cv2.VideoCapture(video_path)
    fps = cap.get(cv2.CAP_PROP_FPS)
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    nfrm = total_frames - 1

    # Auto-detect frame rate from video if not specified
    if frame_rate is None:
        if fps > 0:
            frame_rate = round(fps)
            print(f"\nProcessing: {os.path.basename(video_path)}")
            print(f" Auto-detected frame rate: {frame_rate} fps ({total_frames} frames)")
        else:
            frame_rate = 30
            print(f"\nProcessing: {os.path.basename(video_path)}")
            print(f" WARNING: Could not detect frame rate, using default: {frame_rate} fps")
            print(f" Override by setting frame_rate = 30 (or your value) in config")
    else:
        print(f"\nProcessing: {os.path.basename(video_path)}")
        print(f" Video FPS: {fps}, using override: {frame_rate} fps ({total_frames} frames)")

    # Sanity check: warn if video duration seems unusual
    if frame_rate > 0:
        duration = total_frames / frame_rate
        print(f" Estimated duration: {duration:.1f} seconds")
        if duration < 50 or duration > 70:
            print(f" WARNING: Expected ~60s video but got {duration:.1f}s.")
            print(f" Override by setting frame_rate = 30 (or your value) in config")

```

```

# --- Create background from 100 random frames ---
print(" Calculating background...")
bg_number = min(100, nfrm)
bg_indices = sorted(np.random.choice(nfrm, bg_number, replace=False))

cap.set(cv2.CAP_PROP_POS_FRAMES, 0)
ret, sample = cap.read()
gray_sample = cv2.cvtColor(sample, cv2.COLOR_BGR2GRAY)
bg_array = np.zeros((*gray_sample.shape, bg_number), dtype=np.uint8)

for idx, frame_num in enumerate(bg_indices):
    cap.set(cv2.CAP_PROP_POS_FRAMES, frame_num)
    ret, frame = cap.read()
    if ret:
        bg_array[:, :, idx] = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

background = np.mean(bg_array, axis=2).astype(np.uint8)
bg_scale = 256.0 / (background.astype(np.float64) + 1)
# Pre-crop bg_scale to ROI so per-frame math runs on ROI pixels only
bg_scale_roi = bg_scale[roi_y:roi_y + roi_h, roi_x:roi_x + roi_w]

# --- Process each frame ---
print(" Tracking fly positions...")
threshold = (search_size ** 2) * per_pixel_threshold
half_search = round(search_size / 2)

pos_array = np.zeros((nfrm, 3))

# Seek once; sequential cap.read() is far faster than per-frame seeking
# (compressed formats must decode from the last keyframe on every seek)
cap.set(cv2.CAP_PROP_POS_FRAMES, 0)
for nofr in range(nfrm):
    ret, frame = cap.read()
    if not ret:
        pos_array[nofr] = [nofr, np.nan, np.nan]
        continue

    # Crop to ROI before float conversion so background subtraction
    # operates on ROI pixels only, not the full frame
    frame_roi = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)[
        roi_y:roi_y + roi_h, roi_x:roi_x + roi_w
    ].astype(np.float64)
    frame_div = np.clip(frame_roi * bg_scale_roi, 0, 255).astype(np.uint8)

    # Find fly
    fx, fy = fly_finder(frame_div, half_search, threshold, flip=True)
    pos_array[nofr] = [nofr, fx, fy]

    # Progress update every 10%
    if (nofr + 1) % max(1, nfrm // 10) == 0:
        pct = (nofr + 1) / nfrm * 100
        print(f" {pct:.0f}% complete ({nofr + 1}/{nfrm} frames)")

cap.release()

# --- Convert to real coordinates ---
xscale = width / roi_w
yscale = height / roi_h

corrected_array = np.column_stack([
    pos_array[:, 0] / frame_rate,
    pos_array[:, 1] * xscale,

```



```

        pos_array[:, 2] * yscale,
    ])

    skipped = np.sum(np.isnan(corrected_array[:, 1]))
    print(f" {skipped} points skipped out of {nfrm}.")

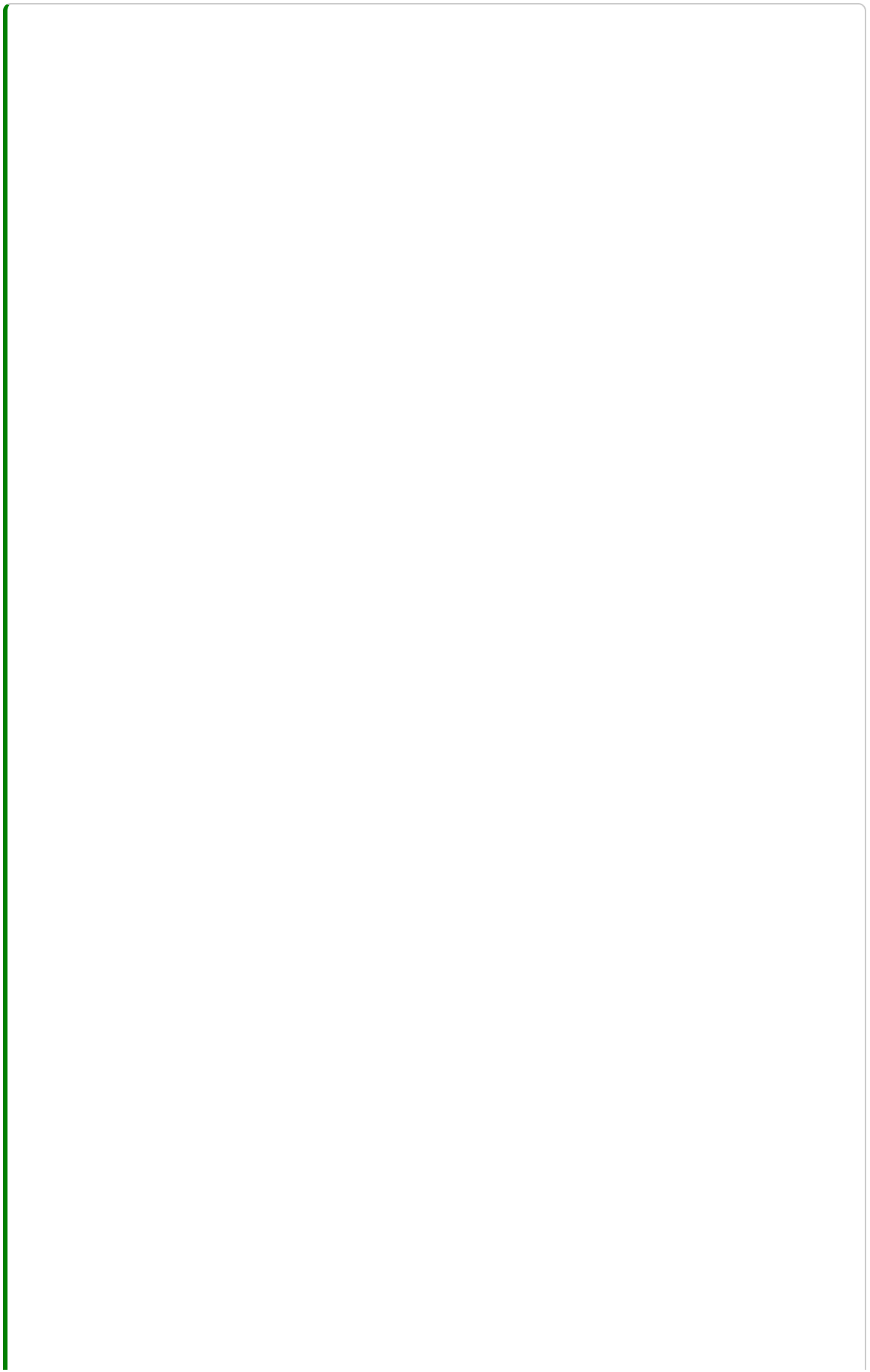
    # Apply filters and interpolation
    corrected_array = boundary_filter(corrected_array, diameter)
    corrected_array = displacement_filter(corrected_array)
    corrected_array = dist_filter(corrected_array, 2)
    corrected_array = interpolate_pos(corrected_array, 2)

    return corrected_array, frame_rate

print('Helper functions defined.')

```

The cell below is going to prompt you to select an ROI using a square. You want to select **the entire petri dish** (you may need to guess where the edges are if they are hard to see).



```

#@title Interactive ROI selector – click and drag on the image { display-mode:

# Read first frame and encode as base64 for display
cap = cv2.VideoCapture(current_video)
ret, frame = cap.read()
cap.release()
if not ret:
    raise ValueError(f'Could not read first frame of {current_video}')

frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
img_h, img_w = frame_rgb.shape[:2]

# Encode frame as PNG base64
_, buf = cv2.imencode('.png', cv2.cvtColor(frame_rgb, cv2.COLOR_RGB2BGR))
img_b64 = base64.b64encode(buf).decode('utf-8')

print(f'Image size: {img_w} x {img_h} pixels')
print('Click and drag on the image below to select the ROI, then click "Confirm

# JavaScript/HTML interactive ROI selector
html_code = f"""
<div id="roi-container" style="position:relative; display:inline-block; cursor:
    <canvas id="roi-canvas" width="{img_w}" height="{img_h}"
        style="max-width:67%; height:auto; border:1px solid #ccc;"></canvas>
</div>
<br>
<button id="roi-confirm-btn" style="font-size:16px; padding:8px 20px; background
    color:white; border:none; border-radius:4px; cursor:pointer; margin-top:8px;">
    Confirm ROI
</button>
<p id="roi-info" style="font-size:14px; color:#333;"></p>

<script>
(function() {{
    var canvas = document.getElementById('roi-canvas');
    var ctx = canvas.getContext('2d');
    var img = new window.Image();
    img.src = 'data:image/png;base64,{img_b64}';

    var drawing = false;
    var x0 = 0, y0 = 0, x1 = 0, y1 = 0;
    var hasROI = false;

    // Scale factor: canvas CSS size vs actual pixel size
    function getScale() {{
        var rect = canvas.getBoundingClientRect();
        return {{ sx: canvas.width / rect.width, sy: canvas.height / rect.height }}
    }}

    img.onload = function() {{
        ctx.drawImage(img, 0, 0);
    }};

    function redraw() {{
        ctx.clearRect(0, 0, canvas.width, canvas.height);
        ctx.drawImage(img, 0, 0);
        if (hasROI) {{
            var rx = Math.min(x0, x1);
            var ry = Math.min(y0, y1);
            var rw = Math.abs(x1 - x0);
            var rh = Math.abs(y1 - y0);

```

```

        ctx.strokeStyle = 'red';
        ctx.lineWidth = 3;
        ctx.strokeRect(rx, ry, rw, rh);
        document.getElementById('roi-info').innerText =
            'ROI: x=' + Math.round(rx) + ', y=' + Math.round(ry) +
            ', w=' + Math.round(rw) + ', h=' + Math.round(rh);
    }}
}}

canvas.addEventListener('mousedown', function(e) {{
    var rect = canvas.getBoundingClientRect();
    var scale = getScale();
    x0 = (e.clientX - rect.left) * scale.sx;
    y0 = (e.clientY - rect.top) * scale.sy;
    drawing = true;
    hasROI = false;
}});

canvas.addEventListener('mousemove', function(e) {{
    if (!drawing) return;
    var rect = canvas.getBoundingClientRect();
    var scale = getScale();
    x1 = (e.clientX - rect.left) * scale.sx;
    y1 = (e.clientY - rect.top) * scale.sy;
    hasROI = true;
    redraw();
}});

canvas.addEventListener('mouseup', function(e) {{
    drawing = false;
}});

document.getElementById('roi-confirm-btn').addEventListener('click', function
    if (!hasROI) {{
        document.getElementById('roi-info').innerText =
            'No ROI drawn yet! Click and drag on the image first.';
        return;
    }}
    var rx = Math.round(Math.min(x0, x1));
    var ry = Math.round(Math.min(y0, y1));
    var rw = Math.round(Math.abs(x1 - x0));
    var rh = Math.round(Math.abs(y1 - y0));

    // Send ROI back to Python via Colab's kernel eval
    google.colab.kernel.invokeFunction('set_roi', [rx, ry, rw, rh], {});
    document.getElementById('roi-info').innerText =
        'ROI CONFIRMED: x=' + rx + ', y=' + ry + ', w=' + rw + ', h=' + rh;
    document.getElementById('roi-confirm-btn').style.background = '#888';
    document.getElementById('roi-confirm-btn').innerText = 'ROI Confirmed!';
    }});
}}());
</script>
"""

# Register a callback so JavaScript can send the ROI to Python
roi_result = {'value': None}

def set_roi(x, y, w, h):
    roi_result['value'] = (int(x), int(y), int(w), int(h))
    print(f'ROI received: x={int(x)}, y={int(y)}, w={int(w)}, h={int(h)}')

output.register_callback('set_roi', set_roi)

```

```
display(HTML(html_code))
```

```
# Run this cell AFTER you have confirmed your ROI above
roi = roi_result['value']

if roi is None:
    raise ValueError("No ROI selected! Go back and draw a rectangle on the image")

print(f'Using ROI: x={roi[0]}, y={roi[1]}, width={roi[2]}, height={roi[3]}')
```

Track Fly in Video(s)

This processes each video frame-by-frame:

1. Creates a background image from 100 random frames
2. Subtracts the background from each frame
3. Finds the fly position using center-of-mass of dark pixels
4. Applies teleportation filter and interpolation
5. Converts pixel positions to centimeters

```
# --- Process the current video ---
corrected, detected_fps = process_video(current_video, roi, diameter, frame_rate,
                                       search_size, per_pixel_threshold)

all_corrected.append(corrected)
all_fps.append(detected_fps)
all_video_paths.append(current_video)

print(f'\nDone! Total videos processed: {len(all_corrected)}')
print('To process another video, go back to "Select Video File" cell and change')
```

Plot Fly Path(s)

Color-coded by time (light blue = start, dark purple = end).

```

for idx, corrected in enumerate(all_corrected):
    x = corrected[:, 1]
    y = corrected[:, 2]
    t = corrected[:, 0]

    fig, ax = plt.subplots(figsize=(6, 6))

    # Plot path colored by time
    valid = ~np.isnan(x) & ~np.isnan(y)
    points = np.array([x[valid], y[valid]]).T.reshape(-1, 1, 2)
    segments = np.concatenate([points[:-1], points[1:]], axis=1)

    lc = LineCollection(segments, cmap='BuPu', linewidth=2)
    lc.set_array(t[valid][:-1])
    ax.add_collection(lc)

    ax.set_xlim(0, diameter)
    ax.set_ylim(diameter, 0) # Invert y to match video
    ax.set_aspect('equal')
    ax.set_xlabel('X-coordinate (cm)', fontsize=11)
    ax.set_ylabel('Y-coordinate (cm)', fontsize=11)
    ax.set_title(f'Fly Path - Video {idx + 1}')

    cbar = fig.colorbar(lc, ax=ax, orientation='horizontal', pad=0.1)
    cbar.set_label('Time (s)')

    plt.tight_layout()
    plt.show()

```

Calculate & Plot Velocity



```

# Calculate velocity for videos that haven't been processed yet
start_idx = len(all_velocity) # Only calculate for newly added videos

for idx in range(start_idx, len(all_corrected)):
    corrected = all_corrected[idx]
    x = corrected[:, 1]
    y = corrected[:, 2]

    fps = round(1.0 / corrected[1, 0]) if corrected[1, 0] > 0 else 30
    total_time = len(x) / fps
    total_bins = int(np.floor(total_time / bin_size))

    # Calculate velocity per bin
    data_rate = round(1.0 / corrected[1, 0]) * bin_size if corrected[1, 0] > 0
    data_rate = int(data_rate)

    if data_rate < 1:
        raise ValueError('bin_size is smaller than the minimum data rate.')

    velocity = np.zeros(total_bins)
    for row in range(0, len(corrected) - data_rate, data_rate):
        bin_idx = row // data_rate
        if bin_idx >= total_bins:
            break
        p1 = corrected[row, 1:3]
        p2 = corrected[row + data_rate, 1:3]
        if np.any(np.isnan(p1)) or np.any(np.isnan(p2)):
            velocity[bin_idx] = np.nan
        else:
            # 10x converts cm to mm
            velocity[bin_idx] = 10.0 * euclidean(p1, p2)

    # Convert from mm/bin to mm/s
    velocity = velocity / bin_size

    all_velocity.append(velocity)

    # Warn about absurd velocities
    if np.nanmax(velocity) > 30:
        print(f'WARNING (Video {idx + 1}): Absurdly high velocities detected.')
        print('  Consider changing the ROI or re-recording the video.')

    mean_vel = np.nanmean(velocity)
    std_vel = np.nanstd(velocity)
    print(f'\n--- Video {idx + 1} (from: {os.path.basename(current_video)}) ---')
    print(f'  Mean velocity: {mean_vel:.2f} mm/s')
    print(f'  Std deviation: {std_vel:.2f} mm/s')

    # Plot velocity over time
    time_axis = np.arange(total_bins) * bin_size
    fig, ax = plt.subplots(figsize=(10, 4))
    ax.plot(time_axis, velocity, linewidth=1.5)
    ax.set_xlim(0, time_axis[-1] + bin_size if len(time_axis) > 0 else 1)
    ax.set_ylim(0, np.nanmax(velocity) * 1.5 if np.nanmax(velocity) > 0 else 1)
    ax.set_xlabel('Time (s)', fontsize=11)
    ax.set_ylabel('Velocity (mm/s)', fontsize=11)
    ax.set_title(f'Fly Velocity - Video {idx + 1}')
    plt.tight_layout()
    plt.show()

```



```
print(f'\nVelocity analysis complete. Total videos analyzed: {len(all_velocity)}
```

Summary Across All Videos

```
num_files = len(all_velocity)

if num_files > 1:
    # Pad velocity arrays to the same length for comparison
    max_len = max(len(v) for v in all_velocity)
    velocity_matrix = np.full((num_files, max_len), np.nan)
    for i, v in enumerate(all_velocity):
        velocity_matrix[i, :len(v)] = v

    # Plot all velocities together
    fig, ax = plt.subplots(figsize=(10, 5))
    time_axis = np.arange(max_len) * bin_size
    for i in range(num_files):
        ax.plot(time_axis, velocity_matrix[i], linewidth=2,
                label=f'Fly {i + 1}')
    ax.set_xlabel('Time (s)', fontsize=11)
    ax.set_ylabel('Velocity (mm/s)', fontsize=11)
    ax.set_title('All Fly Velocities')
    ax.legend()
    plt.tight_layout()
    plt.show()

    # Summary stats across videos
    per_video_means = [np.nanmean(v) for v in all_velocity]
    mean_across = np.mean(per_video_means)
    sd_across = np.std(per_video_means)
    print(f'\n=== Summary Across {num_files} Videos ===')
    print(f'Mean velocity across videos: {mean_across:.2f} mm/s')
    print(f'SD of mean velocity across videos: {sd_across:.2f} mm/s')
else:
    mean_vel = np.nanmean(all_velocity[0])
    sd_vel = np.nanstd(all_velocity[0])
    print(f'\n=== Summary (1 Video) ===')
    print(f'Mean velocity: {mean_vel:.2f} mm/s')
    print(f'SD of velocity: {sd_vel:.2f} mm/s')
```

Prepare for Another Video (Optional)

If you want to process another video with the same ROI:

1. Go back to the **Select Video File** cell and update `current_video` to the new file path
2. Re-run the video selection cell
3. Skip the ROI selection (you can reuse the same ROI with `roi = roi` or draw a new one)
4. Run the **Track Fly** cell again — results automatically append to `all_corrected`

Or, to apply a **different ROI** for the next video, re-run the ROI selection cells before tracking.

Download Results (Optional)

Download the tracking data as CSV files.

```
for idx, (corrected, vel) in enumerate(zip(all_corrected, all_velocity)):
    base = os.path.splitext(os.path.basename(all_video_paths[idx]))[0]

    csv_name = f'{base}_tracking.csv'
    np.savetxt(csv_name, corrected, delimiter=',',
               header='Time_s,X_cm,Y_cm', comments='')
    files.download(csv_name)
    print(f'Downloaded {csv_name}')

    time_axis = np.arange(len(vel)) * bin_size
    vel_csv = f'{base}_velocity.csv'
    np.savetxt(vel_csv, np.column_stack([time_axis, vel]), delimiter=',',
               header='Time_s,Velocity_mm_per_s', comments='')
    files.download(vel_csv)
    print(f'Downloaded {vel_csv}')
```

Measuring Alpha Wave Activity Using EEG

Background & Goals

The cerebral cortex contains huge numbers of neurons. Activity of these neurons is — to some extent — synchronized in regular firing rhythms. These are referred to as brain waves. Electrodes placed in pairs on the scalp can pick up variations in electrical potential that derive from this underlying cortical activity. This recording of the electrical activity in the cortex is called an electroencephalogram (EEG). EEG signals are strongly affected by different states of arousal and show characteristic changes in different stages of sleep. EEG signals are also affected by stimulation from the external environment and brain waves can become entrained to external stimuli. Electroencephalography is used, among other things, in the diagnosis of epilepsy and the diagnosis of brain death.

Recording EEG signals

EEG recording is technically difficult, mainly because of the small size of the voltage signals, which are typically 50 μV **peak-to-peak**. The signals are small because the recording electrodes are separated from the brain's surface by the scalp, the skull, and a layer of cerebrospinal fluid. A specially designed amplifier, such as the Bio Amp built into the PowerLab, is essential to record EEGs. It is also important to use electrodes made of the right material and to connect them properly. Even with these precautions, recordings may be spoiled by a range of unwanted interfering influences, known as artifacts.

In this lab, you will record EEG activity with an occipital electrode on the scalp at the back of the head. A third (ground or earth) electrode is also attached, to reduce electrical interference. In clinical EEG, it is usual to record many channels of activity from multiple recording electrodes placed in an array over the head.

Origins of the EEG signals

The EEG results from slow changes in the membrane potentials of cortical neurons, especially the excitatory and inhibitory post-synaptic potentials (EPSPs and IPSPs). Very little contribution normally comes from action potentials propagated along nerve axons. The EEG reflects the sum of the electrical potential changes occurring from large populations of cells. Therefore, large amplitude waves require the synchronous activity of a large number of neurons. The rhythmic events that these waves reflect often arise in the thalamus whose activity is in turn affected by a variety of inputs including structures in the brainstem reticular formation.

Components of the EEG waveform

The EEG waveform contains component waves of different frequencies. These can be extracted and provide information about different brain activities. The types of brain waves are:

- **alpha** (between 8 to 13 Hz; average amplitudes of 30 to 50 μV peak-to-peak), which will be studied in this experiment. Alpha rhythm is seen when the eyes are closed and the volunteer relaxed. It is abolished by eye opening and by mental effort such as doing calculations or concentrating on an idea. It is thus thought to indicate the degree of cortical activation. The greater the activation, the lower the alpha activity. Alpha waves are strongest over the occipital (back of the head) cortex and also over the frontal cortex.
- **beta** (13 to 30 Hz; <20 μV peak-to-peak), which are prominent in alert individuals with their eyes open. The beta rhythm may be absent or reduced in areas of cortical damage

and can be accentuated by sedative-hypnotic drugs such as benzodiazepines and barbiturates.

- **theta** (4 to 8 Hz; <30 μ V peak-to-peak), which are seen in awake children but not adults. The theta rhythm is normal during sleep at all ages. However, some researchers separate this frequency band into two components: low theta (4–5.45 Hz) activity correlated with decreased arousal and increased drowsiness, and high theta (6–7.45 Hz) activity that is claimed to be enhanced during tasks involving working memory.
- **delta** (0.5 to 4 Hz; up to 100–200 μ V peak-to-peak), which is the dominant rhythm in sleep stages three and four but not seen in conscious adults. The delta rhythm tends to have the highest amplitude of any of the component EEG waves. EEG artifacts caused by movements of jaw and neck muscles can produce waves in the same frequency band.
- **gamma** (30 to 50 Hz). Most people recognize gamma rhythm, but its importance is controversial. It may be associated with higher mental activity, including perception and consciousness, and it disappears under general anesthesia. One suggestion is that the gamma rhythm reflects the mental activity involved in integrating various aspects of an object (color, shape, movement, etc.) to form a coherent picture.

Written by staff of ADInstruments & A. Juavinett. Copyright © 2015 ADInstruments Pty Ltd. All rights reserved. PowerLab® and LabChart® are registered trademarks of ADInstruments Pty Ltd.

Lab Protocol

Supplies

- LabChart software & computer
- PowerLab 26T
- 5 Lead Shielded Bio Amp Cable
- EEG Flat Electrodes (gold contacts)
- Electrode paste
- Abrasive Gel (NuPrep)
- Alcohol swabs
- Medical tape
- Elastic bandage
- Cotton swab

Part I. Spectral Analysis Tutorial

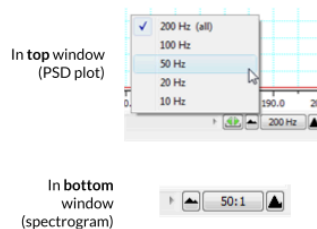
A spectrum is a representation of data based on the frequency distribution of its component sine waves. Spectra indicate the strength of the various frequencies in a time-varying waveform. Spectrum View allows you to observe the frequency distribution of data that might not otherwise be easily seen. For example, it could be used to break down an EEG waveform into its various components: beta waves, alpha waves, theta waves and delta waves. A mathematical technique known as the Fast Fourier Transform is applied to the raw data. The results of this analysis can be displayed as a plot of the power (vertical axis) of different frequencies (horizontal axis) relative to each other in the input signal. This is called a **Power Spectrum Density (PSD) plot**. The data can also be displayed as a 3-dimensional color plot of spectral power, frequency, and time, called a **Spectrogram**.

Note: The questions here are written to ensure that you are understanding the steps of the tutorial. You do not need to answer them, and they do not directly relate to your lab report. You *should*, however, understand how to interpret a spectrogram.

1. Open the EEG Spectral Analysis Tutorial settings file (<http://bit.ly/labchart>).
2. Examine the Chart View. Use the **View Buttons** to view each block. You should see five blocks of data. The first record is a slowly oscillating sine wave.
3. Open Spectrum view by clicking on the Spectrum View button in the Toolbar:



4. Click the **Smart Tile** button in the LabChart Toolbar to display both windows in full screen mode.
5. In Chart View, **select** the first record by double-clicking in the Time axis. This will perform a spectral analysis for this record and display the result in the Spectrum view.
6. Adjust the horizontal scaling of plots to view the results:
 - Set the **horizontal scaling** for the **Power Spectrum Density (PSD)** plot to 50 Hz.
 - Use the horizontal scroll bar to display the 0 Hz to 50 Hz region of the plot.
 - You may need to Autoscale the axes.
 - Set the **horizontal scaling** for the Spectrogram to 50:1.



7. Examine the PSD plot and then the first section of the Spectrogram. Expand the vertical axes if necessary. Use the waveform cursor to identify the frequency in Hertz (Hz) of the peak in the PSD plot and the band in the Spectrogram. Values are displayed at the top of each plot.
8. Select the second record and again view the result in the Spectrum view.

9. Select the third record and again view the result in the Spectrum view. You should now see two prominent peaks (PSD plot) and bands (Spectrogram) in the result.
10. Select the fourth record and again view the result in the Spectrum view.

Questions for reflection

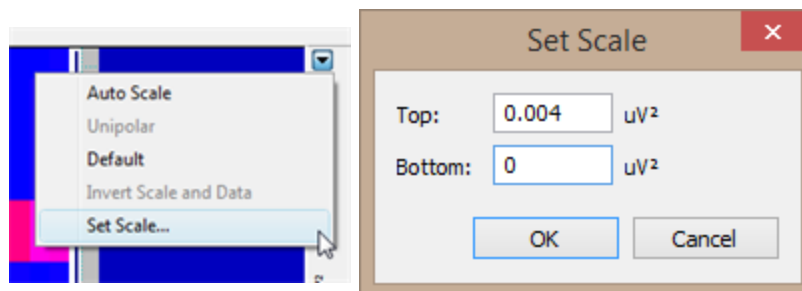
- What is the frequency in Hertz (Hz) of the first sine wave?
- What is the frequency in Hertz (Hz) of the second sine wave?
- Are the two peaks/bands you see in record 3 the same as for the first two records? How is this possible?
- Is there any regular signal within record 4?

11. In Chart View compare the signal amplitudes of the fourth and fifth records. Note that the fifth record has lower amplitude compared with the fourth record (you may need to Autoscale to see a peak at all).
12. Select the fourth record again. In the Spectrum view examine the PSD plot. Move the **Waveform Cursor** to the prominent peak.
13. Select the fifth record and examine the PSD plot. Move the **Waveform Cursor** to the peak.

Questions for reflection

- What is the frequency (Hz) of the signal in record 4?
- What is the power (mV^2) of the signal in record 4?
- What is the frequency (Hz) of the signal in record 5?
- What is the power (mV^2) of the signal in record 5?

14. Examine the Spectrogram. Note that the band corresponding to the signal's frequency appears to be missing. This is because the power of the signal is small compared with the previous four records. Expand the scale on the right-hand side of the Spectrogram by setting the scale closer to the power of the signal (e.g., right-click **Set Scale** to 0 to 0.004 V^2). Note that the band is now visible at the expected frequency.



15. The fifth record is the same signal as the fourth record, except that the quality of the raw signal has been affected. Compare your features (amplitude, power, frequency) of the fourth and fifth records.

Questions for reflection

- How has the quality of the signal affected the wave features?

Part II. Detecting alpha waves

Equipment Setup and Electrode Attachment

1. Make sure the PowerLab is turned **off** and the USB cable is connected to the computer.
2. Connect the 5 Lead Shielded Bio Amp Cable to the Bio Amp Connector on the front panel of the PowerLab (Figure 1). The hardware needs to be connected **before** you open the settings file.

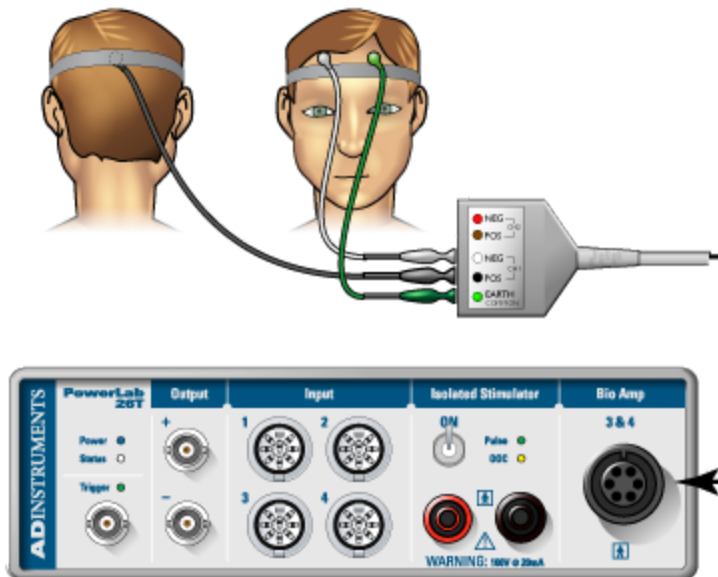


Figure 1. Equipment setup for PowerLab 26T. Image courtesy of ADInstruments ©

3. Attach the leads of the EEG Flat Electrodes to the Earth, CH1 NEG and POS pins closest to the labeled side on the Bio Amp Cable. Channel 1 “**positive**” will lead to the inion (the bump on the back of the head above the neck) and Channel 1 “**negative**” will lead to the forehead. Channel 2 will be empty and the Earth will lead to the temple. Refer to Figure 1 for proper placement, but do not attach them to the volunteer yet. Follow the color scheme on the Bio Amp Cable.
4. Remove any jewelry from the volunteer’s face, ears, and neck.
5. With Figure 1 as a guide, locate where you will place the electrodes. Abrade the skin with the NuPrep Abrasive Gel.
Note: This is important as abrasion helps reduce the skin’s resistance. Your recordings will be much better if you have abraded the skin well (but not too much — that will cause irritation).
6. After abrasion, clean the area with an alcohol swab to remove the dead skin cells.
7. While the skin is drying, scoop Electrode Paste into the EEG Flat Electrodes. When the skin is dry, stick the electrodes to the skin (Figure 1). Immediately hold the electrodes and wires in place with the medical tape. Make sure the tape firmly holds the electrodes

- against the head. Use the elastic bandage to wrap tightly around the head. This will help the electrodes maintain good contact with the skin.
8. Have the volunteer lie in a comfortable position on his/her back (or sit comfortably), with the head turned so that none of the electrodes are disturbed or compressed.
 9. Check that all three electrodes are properly connected to the volunteer and the Bio Amp Cable before proceeding. Turn on the PowerLab.

Exercise 1: Recognizing Artifacts

In this exercise, you will learn to recognize some of the artifacts that can appear on an EEG.

1. Launch LabChart with the EEG Settings file (<http://bit.ly/labchart>).
2. Select **Bio Amp** from the EEG Channel Function pop-up menu. Make sure the settings are as follows: **Range 200 μ V, High Pass 0.5 Hz, and Low Pass 50 Hz.**

Questions for reflection

- Why are we applying high- and low-pass filters to our data?

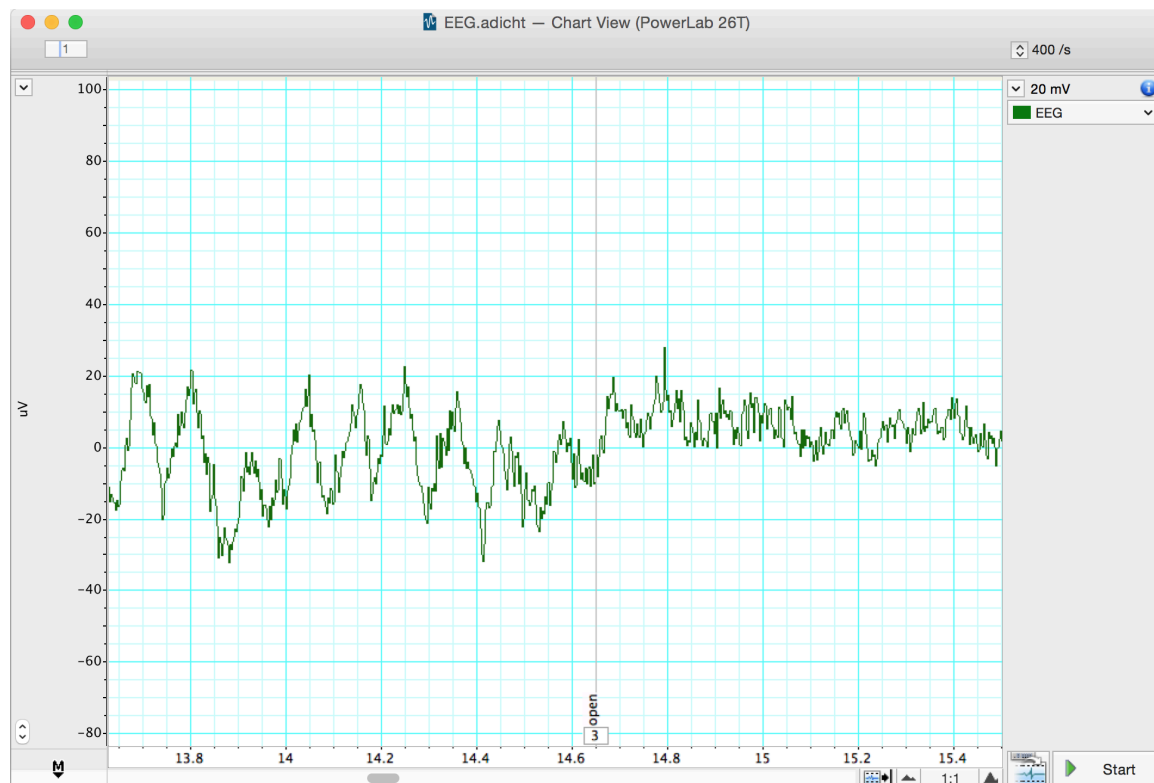
3. **Start** recording. Add a **comment** "blinking," and have the volunteer blink repeatedly. **Stop** recording after 10 seconds.
4. Repeat step 3, this time, have the volunteer make eye movements. Add a **comment** "eye movements." Have the volunteer gaze up-and-down and left-and-right in a repeated pattern. Make sure the volunteer's head is still and only the eyes move.
5. Repeat step 3, this time, have the volunteer make head movements. Add a **comment** "head movements." Have the volunteer gently move his/her head in a repeated pattern.
6. Examine the vertical scale at the left of the Chart View, and note the positions corresponding to +50 μ V and -50 μ V. At rest (when your volunteer is not blinking or moving their head), the EEG recording should not exceed these limits. It is okay if it goes beyond -50 μ V or above 50 μ V when the person is blinking or moving. If your signal exceeds these limits, please ensure that the electrode on the back of the head is making electrical contact with the scalp and reposition if necessary.
7. **Save your data**, and open a new file with the same settings.

★ Exercise 2: Alpha Waves in the EEG

In this exercise, you will examine the effects of relaxation and eye movement on alpha waves in the EEG.

1. Make sure the volunteer is relaxed and comfortable. Have the volunteer close his/her eyes and remain quiet. Keep noise to a minimum and keep all distractions away from the volunteer.
2. **Start** recording. Prepare a comment with "open"; do not enter it yet.
3. Tell the volunteer to open both eyes. Immediately press Return/Enter to add the **comment**. Record with the volunteer's eyes open for 30 seconds. Do not stop recording.

4. Prepare a comment with "shut." When the 30 seconds are complete, tell the volunteer to close both eyes. Immediately press Return/Enter to add the **comment**.
5. Repeat steps 3 and 4 nine times to give you ten total repeats of each condition. **Save your data.**



EEG signal with eyes shut and open. Note alpha waves during eyes shut (left).

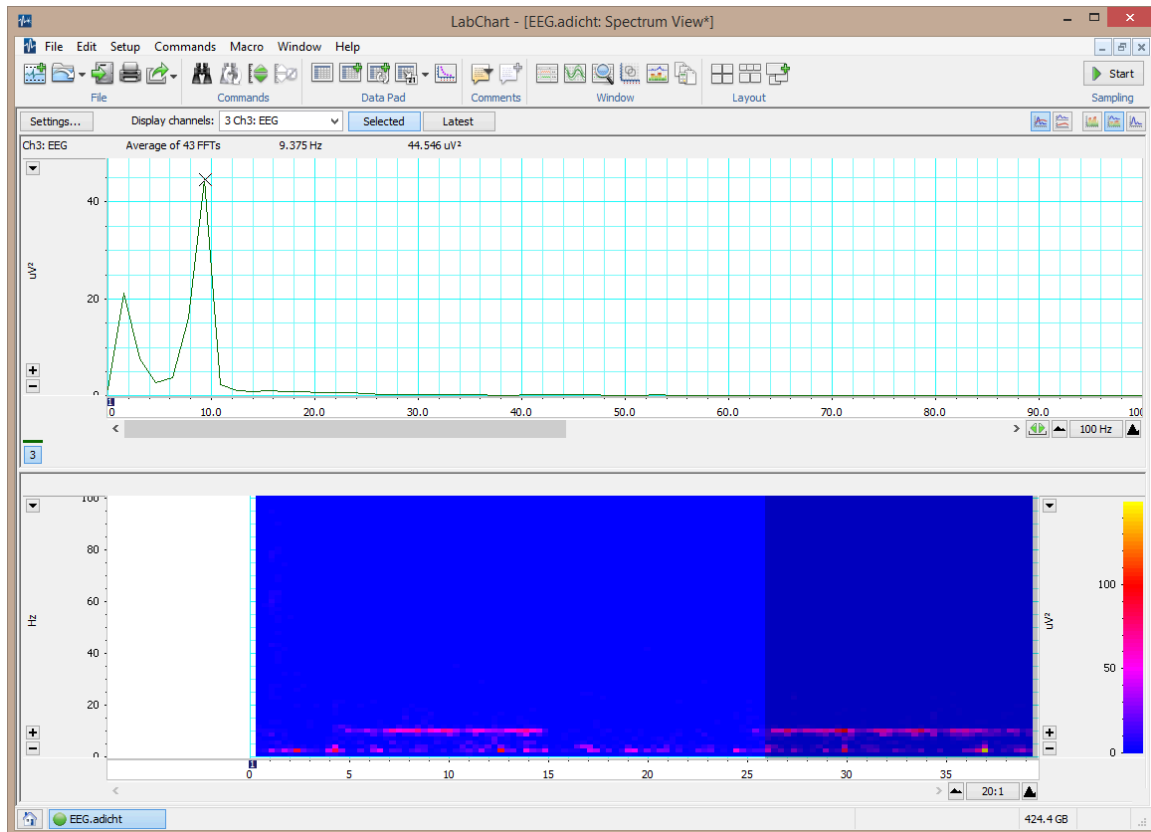
Wave Amplitude Analysis

1. Use the **View Buttons** to change the horizontal compression to see data with eyes open and shut. Make a data selection that includes some data from both eyes open and eyes shut conditions. View this selection in **Zoom View**. This should make it easier to see the alpha wave activity. **Autoscale**, if necessary.
2. In Chart View, scroll through the parts of the recording that were made with the volunteer's eyes shut to look for alpha waves. Use the **View Buttons** to change the horizontal compression if necessary. The alpha waves can be recognized by their amplitude (usually 30 to 50 μV peak-to-peak) and their frequency. Each cycle of an alpha wave lasts ~ 0.1 s.
3. Use the **Marker** and **Waveform Cursor** to measure the amplitude of the alpha waves. Place the **Marker** at the lowest point of the wave and move the **Waveform Cursor** to the peak of the wave. Measure the amplitudes of five waves from when the volunteer's eyes were closed. Record the values in **Table 1** of the Data Notebook.
4. Now measure wave amplitudes when the volunteer's eyes were open. Record these values in **Table 2** of the Data Notebook.

Spectral Analysis

Now you will use Spectral Analysis to examine the EEG you recorded.

1. Find the part of the recording when the volunteer had their eyes **shut**. Click-and-drag across this part of the data trace to **select** it.
2. From the **Window** menu, select **Spectrum**. In the Spectrum View, choose **Selected** (underneath the main toolbar at the top).



Spectrum of an EEG

3. Next to Display Channels, choose Ch 3: EEG.
4. Alpha activity should show up in the PSD plot as a clear peak in the 8–12 Hz range.
5. Alpha activity shows up in the Spectrogram as a band of color in the 8–12 Hz range. If you cannot see the alpha activity as a clear peak in the 8–12 Hz range, manually scale the horizontal and vertical axes. Note that Spectrogram displays all the recorded data and that the selection you have made is highlighted in a darker blue color.
6. Make a data selection of several seconds from when the volunteer had their eyes **open**. Select **Spectrum**. Note that in the PSD plot the peak in the alpha activity range of 8–12 Hz is small or absent, and in the Spectrogram the band of color in the alpha activity range of 8–12 Hz is weak or absent.
7. In the Spectrogram, scale the horizontal axis so that all the data is visible. Note the presence and absence of the band of color in the alpha activity range of 8–12 Hz, which corresponds with the eyes shut and eyes open conditions.
8. If you're happy with your spectrograms (e.g., you can clearly see alpha activity in closed conditions but not open conditions), you've completed the task for this lab. Save your data in a cloud location.



Questions for reflection

- Under what conditions did you see alpha waves more clearly?
- What are alpha waves thought to indicate?
- Look at your Spectral Analysis from when the volunteer had their eyes closed. At what range do you have the greatest peak? What type of brain waves does the Spectral Analysis suggest is active?

Analyzing your EEG data

1. Export 30 seconds of one closed trial and one open trial for your analysis. [See this page if you need a reminder.](#)
2. Using this notebook, you can generate plots from your EEG data: <https://bipn145.github.io/Python/SpectralAnalysis.html>

Data Notebook

Table 1. Alpha Waves with Eyes Open and Closed

	Eyes Open Amplitude (μV)	Eyes Closed Amplitude (μV)
Wave #1		
Wave #2		
Wave #3		
Wave #4		
Wave #5		
Wave #5		
Wave #6		
Wave #7		
Wave #8		
Wave #9		
Wave #10		